Selection-Aware Queue Forecasting and Noise-Floor State Reconstruction for TrafficFlowBench 2026

Lukas Buck
Lukas.Buck@reutlingen-university.de
Reutlingen University, School of Informatics, Reutlingen, Germany
SixSentences Research, Stuttgart, Germany
Kaggle team "Lukas" · https://github.com/L4XB/trafficflowbench-solution

Abstract

TrafficFlowBench poses four problems on simulated freeway corridors: reconstruct hidden detector values (Task 1) consistently with traffic physics (Task 3), forecast which links will be queued in the next 30 minutes (Task 2), and estimate path flows from link counts (Task 4). A U-Net ensemble reconstructs the speeds close to a detector-noise floor that we measure directly, and for Task 4 we compute the organisers' published estimator exactly. Most of our work went into Task 2, whose windows are placed by a documented selection rule. Onset queues therefore appear at the last horizon step, and the reason a window was chosen predicts how its queue moves; onsets also recur in a few fixed link blocks. We encode these facts as decoding rules on top of gradient-boosted cell classifiers that never see data after the forecast origin, and we measured every adopted change as a paired difference on the public leaderboard. Both final submissions score 0.90985 there, and Final B scores 0.89721 on the private (April) leaderboard, rank 10 of 802. On April every adopted change with local support gained again, while the small steps chosen on the public board alone added almost nothing. One script rebuilds both finals on a CPU, and cutting the release at any of ten window origins changes no forecast.

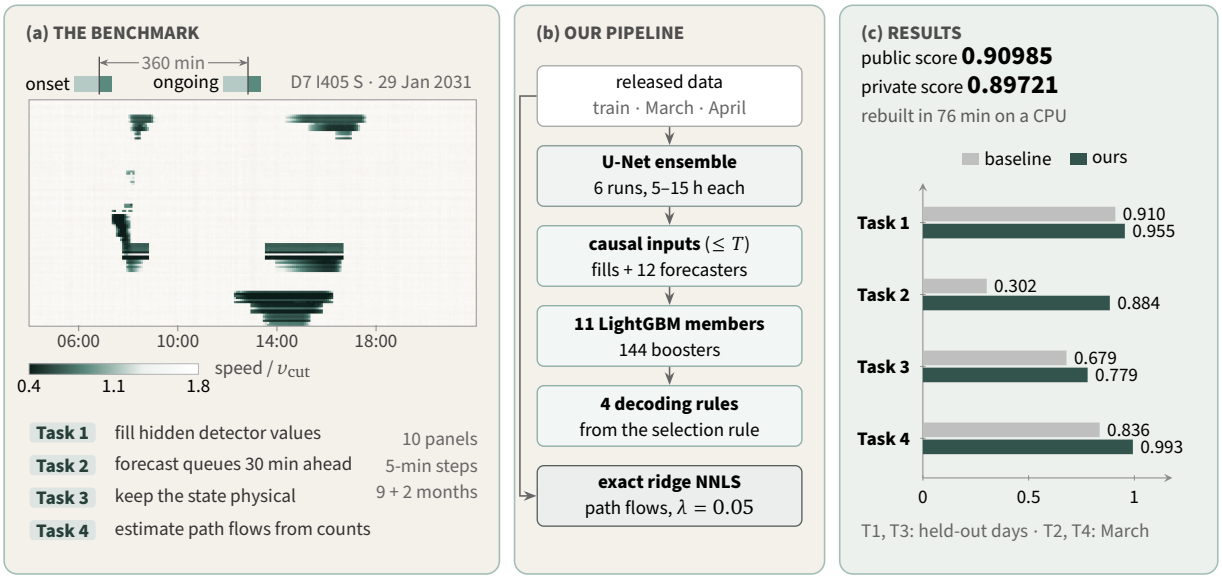




Figure 1 | **Four tasks on one corridor system, one pipeline, and a gain over the baseline on every task.** (a) One training corridor-day: speed relative to the queue cut-off (dark = queue) and the two Task-2 windows that the selection rule places on it. (b) Our pipeline. (c) Scores against simple baselines: linear interpolation for Tasks 1 and 3 on 31 held-out train days (the evaluation months have no labels), the organisers' persistence and count baselines [18] for Tasks 2 and 4 on the March leaderboard.

1. Introduction

Traffic data are never complete. Detectors fail, values arrive late, and operators still need to know what the traffic is doing now and what it will do next. TrafficFlowBench [18] turns this into four tasks on one simulated freeway system. The data come from a calibrated traffic-flow model that follows first-order traffic physics [7, 12, 16], with realistic detector noise and outages on top. Three tasks are *offline*: reconstruct the hidden state (Task 1), keep it physically consistent (Task 3), and estimate path flows from link counts (Task 4). One task is *online*: forecast the queues of the next 30 minutes from the past hour only (Task 2). Figure 1a shows the kind of data involved, one corridor over one day.

The tasks reward different things. The offline tasks reward careful estimation, and there is little to gain once an estimate is as good as the detector noise allows. The online task rewards an understanding of the benchmark itself. Its windows are not random: a documented rule decides where they are placed, and that rule shapes what a good forecast looks like. Most of our work, and most of our score gain, came from taking this rule seriously.

Our main contributions are:

- **State reconstruction at the noise floor.** Six U-Nets, each trained for 5 to 15 hours, fill the hidden cells. A gradient-boosted corrector repairs the long gaps that follow each queue window. Each link's free-flow level in the month corrects the free-flow cells. We measure the detector-noise floor of the speeds from the data and find a single network's error 3–19 % above it. Training longer mattered more than averaging more models.
- **Queue forecasts that model the selection rule.** We use labels chosen with the selector as an oracle, training windows drawn by a replica of the selector, inputs that never see data after the forecast origin, and eleven gradient-boosted cell classifiers. Decoding uses four rules: expected-IoU decoding, a live-signal gate, a snap to the corridor's onset blocks, and thresholds that follow the reason a window was chosen. Kinematic waves explain why a short onset block fills within one five-minute step, and the generator gives no early warning of a breakdown, unlike the real detector data quoted in its configuration (Sections 4.3.4 and 6.4).
- **The exact path-flow estimator.** The Task-4 problem is under-determined: every panel has 18 to 51 times more paths than independent counts. We solve the organisers' published estimator exactly and show that its prior decides most of the answer.
- **A measurement protocol for a small leaderboard.** With 80 scored windows per month, a single score is noisy. Paired differences, block ablations and a forward test in time show which local gains transfer to a new month, and many do not. The private board shows where the protocol ends: paired differences of 0.002 or more kept their sign on April in 18 of 20 cases, smaller ones only by chance.
- **Reproducible and causal by construction.** One script rebuilds both final submissions on a CPU. A truncation test reruns the Task-2 pipeline on copies of the release that end at a window's origin; it found, and helped remove, a hidden dependence between windows.

Both final submissions score 0.90985 on the public (March) leaderboard, up from 0.87063 for our first complete pipeline. About +0.026 of the gain came from Task 2, +0.0072 from Task 4 and



Table 1 | **The three splits of the release.**

split	dates	days	unmasked observations	leaderboard
train	2030-06-01 to 2031-02-28	273	yes	—
validation	2031-03 (March)	31	no	public
private	2031-04 (April)	30	no	private (final ranking)

+0.0061 from the state model. Final A uses general rules only; Final B adds one rule that acts on April windows only. On the private (April) leaderboard, revealed after the deadline, Final A scores 0.89715 and Final B 0.89721; Kaggle counts Final B, rank 10 of 802 teams. The same 21 steps from the first pipeline gained +0.0253 there, against +0.0392 on March (Section 6.6). Figure 1c compares each task with a simple baseline: Tasks 1 and 3 on 31 held-out train days, because the evaluation months have no labels, and Tasks 2 and 4 on the March leaderboard.

Related work. Traffic-state imputation has moved from interpolation and tensor completion to neural imputers such as BRITS [4] and the graph-based GRIN [6]; our U-Nets take the image view of a corridor-day instead. Short-term traffic forecasting is dominated by spatio-temporal graph networks [11, 20], while breakdown at freeway bottlenecks is classically modelled as a stochastic capacity event [3]. Path-flow estimation from link counts goes back to generalised least squares [1, 5]. How adaptive submissions overfit a leaderboard is analysed by Blum and Hardt [2] and Dwork et al. [8]; our paired differences and block ablations are a practical answer.

The rest of the report is organised as follows. Section 2 describes the benchmark, its score, and the rule that places the Task-2 windows. Section 3 explains how we measured progress. Section 4 presents the pipeline, and Section 5 the results and ablations. Section 6 asks what limits each task and what transferred to a new month. Section 7 covers reproducibility, and Section 8 concludes.

2. TrafficFlowBench

2.1. Data and splits

The release covers ten directional freeway panels, two directions of five corridors in Caltrans districts 7 and 12, at five-minute resolution. Every panel ships mainline detector records, ramp counts, the link network with fundamental-diagram parameters, a path set, and a weak origin–destination prior. Table 1 lists the three splits. Only the train split comes with unmasked observations. The two evaluation months were generated independently, with their own demand draws and incidents (five in March, seven in April). Neither month is a sample of the other.

2.2. Tasks and score

The leaderboard combines the four tasks with fixed weights,

$$S = 0.35\, S_{\text{state}} + 0.30\, S_{\text{queue}} + 0.15\, S_{\text{phys}} + 0.20\, S_{\text{ODME}}, \tag{1}$$

where each component is averaged over the two directions of a corridor and then over the corridors.

Task 1: state. The task asks for speed and flow at blanked cells. Three masking regimes hide 20, 30 and 50 % of the eligible cells, one regime per day. Per regime the score is

$$S^{(r)}_{\text{state}} = 0.54 \left(1 - \frac{\text{RMSE}_v}{25}\right) + 0.46 \left(1 - \frac{\text{RMSE}_{q/\text{lane}}}{600}\right), \tag{2}$$

with speed errors in km/h and flow errors in vehicles per hour per lane.



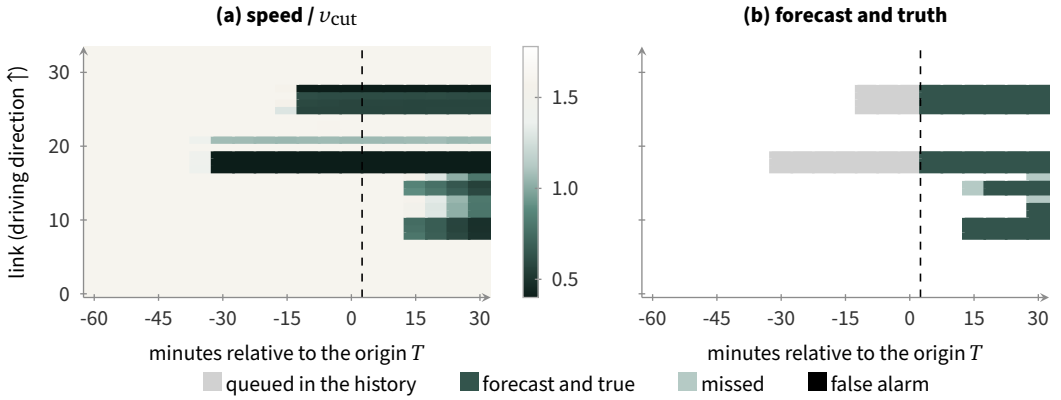


Figure 2 | **A queue grows upstream within one 30-minute horizon.** A Task-2 window in space and time (forward test: models fitted on June–November; window from 4 February 2031, corridor D12 I5 N, origin T = 13:25). The forecast may use the 60 minutes before T and covers the six five-minute steps after it. (a) Speed field: the queue is the dark band, and its tail grows upstream. (b) The six-member forward blend at the base threshold 0.38 against the truth (our label field, Section 4.3.1). The growth of the tail is caught; four late cells at its upstream end are missed. On the whole window, which holds three queues, the forecast reaches an IoU of 0.887.

Task 3: physics. The task has no rows of its own. It rebuilds the state from the Task-1 values, derives the density $k = q/v$ and the vehicle count $N = kL$ of each link, and asks two questions. Do the flows match the fundamental diagram? And does each link conserve vehicles?

$$S_{\mathrm{FD}} = 1 - \frac{\sum |q_{\mathrm{lane}} - q_{FD}(k_{\mathrm{lane}})|}{\sum |q_{\mathrm{lane}}|}, \qquad S_{\mathrm{LWR}} = 1 - \min\Big(1, \frac{\sum |\Delta N - \rho|}{\sum |\rho|}\Big), \tag{3}$$

and $S_{\mathrm{phys}} = \frac{1}{3} S_{\mathrm{FD}} + \frac{2}{3} S_{\mathrm{LWR}}$. Here $\rho = \Delta t\,(q_{\mathrm{in}} + q_{\mathrm{on}} - q_{\mathrm{out}} - q_{\mathrm{off}})$ is the net inflow of a link in one step and q_{FD} the triangular diagram with the released parameters.

Task 2: queues. This is the online task, on eight of the ten panels (all but D12 I405 N and S). Each window publishes 60 minutes of history up to its origin T. The forecast marks the queued links at each of the six steps T+5, …, T+30; a link is queued when its noise-free speed is at most $v_{\mathrm{cut}} = 0.6\,v_f$, with v_f its free-flow speed. A window is *onset* if its history shows no queue and *ongoing* otherwise. For the train split the release also publishes the windows themselves, ten per panel and 80 in all, which lets us check a replica of the selector. Each window is scored by the space-time intersection-over-union (IoU) of the predicted and the true set,

$$\mathrm{IoU}_w = \frac{|\hat{Q}_w \cap Q_w|}{|\hat{Q}_w \cup Q_w|}, \qquad S_{\mathrm{queue}} = \frac{1}{|W|} \sum_{w \in W} \mathrm{IoU}_w, \tag{4}$$

where the month's set W holds five onset and five ongoing windows per panel: 80 windows. One window therefore has a weight of 0.30/80 = 0.00375 in S, which we call a *window unit*. Figure 2 shows one window with our forecast.

Task 4: path flows. The task asks for the flows on all paths of a fixed evening period that best explain the released link counts. Each panel comes with a path set, the 0/1 incidence matrix A between paths and links, the counts c on the counted links $\mathcal{M}$, and a weak prior b on the path flows, all specific to the split. The score compares our path flows $\hat{f}$ with the organisers' reference $f^\star$,

$$S_{\mathrm{ODME}} = 0.45\,S_{od} + 0.25\,S_{link} + 0.15\,S_{dev} + 0.15\,S_{attr}, \tag{5}$$

where S_{od} is one minus the relative L_1 distance to the reference path flows, and S_{link} the same for the link counts. S_{dev} compares how far $\hat{f}$ and $f^\star$ moved away from the prior, and S_{attr} compares the shares of trips per destination.

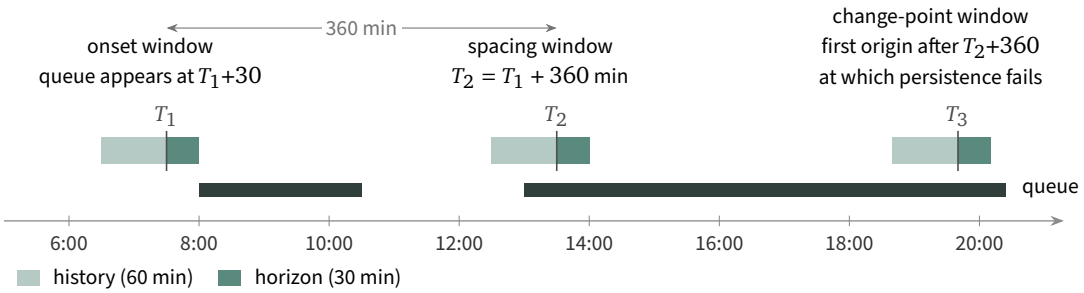


Figure 3 | **The selection rule fixes where each window starts.** How the selector places the Task-2 windows of one panel and day (schematic). The first window that qualifies is an onset window whose queue appears exactly at its last horizon step. The next window can start 360 minutes later at the earliest. If a queue that changes enough exists at that moment, the window is placed by the spacing constraint. Otherwise the next window waits for the first origin at which persisting the current state fails, a change point.

2.3. How the Task-2 windows are chosen

The organisers published the rule that places the windows. Windows of one panel are at least 360 minutes apart. The horizon must contain a queue, and the history must be covered to at least 70 %. An ongoing window is skipped when simply repeating the last state would already reach an IoU above 0.9. The scored windows are the first that qualify in each month: five per condition and panel. The organisers confirmed in the competition forum that "conditioning a forecast on the documented window-selection procedure" is within the rules, subject to the time rule of Section 2.4: a window may use only the published origins at or before its own T [19].

We re-implemented the selector and ran it on the train split, an approach the organisers call reasonable for building train-only windows [19]. Run on candidate label fields, it also tells us which candidate label field comes closest to the organisers' noise-free truth (Section 4.3.1). For training we run the same timeline without the monthly quota, so that every admissible origin of every month contributes a window. Three consequences shape our queue pipeline (Figure 3).

(i) *Onset queues start at T+30.* The earliest admissible origin is taken, so in 39 of the 40 published train onset windows the queue first appears exactly at the last horizon step. An onset forecast is a set of links at one time step.

(ii) *Scored windows sit at the start of the month.* 27 of the 40 March and 25 of the 40 April ongoing windows lie on days 1–4. Features that summarise earlier days of the month are nearly empty there, unlike in most training windows.

(iii) *The origin says why a window was chosen.* An ongoing window that starts exactly 360 minutes after the previous one was placed by the spacing constraint; its queue existed before and is caught in an arbitrary phase. A window that starts later sits at a *change point*, the first origin at which persistence fails. A *young* window has no link with two queued observations before T−5: its queue formed just before the origin.

2.4. The time rule

Task 2 is an online task. For a window with origin T, a forecast may use any released data stamped at or before T: earlier times of the same day, earlier days of the same split, earlier splits and the whole train split. Nothing stamped after T may be used, in any file. This includes the ramp counts inside the horizon and the mainline data that resume after the blank period that follows it. The organisers check the rule by rerunning each finalist's Task-2 pipeline on data cut at the origins [19]. Section 4.3.7 explains how our pipeline keeps the rule and Section 7.3 how we tested it.

3. Evaluation protocol

The evaluation months carry no labels. Kaggle scores every submission on a public leaderboard (March) and on a private one (April, hidden until the end), *boards* for short. The public board scores only 80 Task-2 windows, so we relied on two kinds of evidence: local tests on the train split, and paired differences on the public board.

3.1. Local tests

For Task 1 we held out 31 train days (*holdout31*) and masked them like the evaluation months. For Task 2 we used a forward test in time: fit on June–November, test on December–February, with windows placed by our selector replica (682 onset and 1 535 ongoing windows). A forward test imitates the real situation, in which the models meet a month they have not seen. Only the LightGBM members are refitted. The U-Net fills and forecasts that feed them are out of fold by day but saw all training months, so absolute forward levels are slightly optimistic. We therefore use the forward test for paired comparisons. For the decoding rules we also built a *March replica*. Our selector, run on queue labels from the released March observations, places 648 windows (195 onset and 453 ongoing) on the March data in the same way as the scored ones. The models are fitted on the train split only, so the replica tests a rule on March-like windows without any leaderboard feedback. Its labels come from the same reconstruction that feeds the features, so it can judge decoding rules but not features. For Task 4 we re-implemented the scoring formulas of Equation (5) and predicted the score of a candidate before submitting it (Section 5.4).

3.2. The public leaderboard as a measuring instrument

Of the 207 submissions of our last six weeks, 166 changed one model or rule at a time, 23 were block ablations, 13 were candidates for the second final, and 5 were reproducibility checks. A single score is noisy. The organisers report a per-window IoU standard deviation of 0.375 for persistence; a better forecaster spreads less. With 80 such windows, the level of S carries a sampling error of up to 0.013, more than most of our improvements. Two kinds of submission avoid this noise.

A *paired difference* between two submissions that differ in one block is exact for March: both are scored on the same windows, so the sampling error cancels. As an estimate for April it carries the sampling error of the per-window differences, far smaller than that of a level but not zero, because April has its own windows. A *block ablation* replaces all onset or all ongoing windows of one panel, or of all panels, by an empty forecast. The score then drops by exactly the weight of the block times its level. Zeroing the onset windows of one panel, for instance, costs $0.30 \times 5/80 = 0.01875$ times that panel's onset IoU. Ablations measure levels without revealing anything about single windows. The rules forbid inferring hidden labels by probing the leaderboard window by window [18]; an ablation returns one number for five or more windows, and none served to pick the answer for a single window.

Two cautions follow. A rule picked as the best of several submissions is biased upwards on March (the winner's curse). And March is not April. We therefore adopted only general rules, preferred rules with local support, and recorded for every adopted step whether it rests on a local test, on the score model, or on the leaderboard alone (Table 3). Every output of both finals is produced by code from released data; no output is chosen or edited per window.

The private scores, revealed after the deadline, put this protocol to a test (Section 6.6). Of the 67 distinct paired differences in this report with a March effect of at least 0.0003, the 20 of at least 0.002 kept their sign on April in 18 cases, the 47 smaller ones in 24, no better than chance.



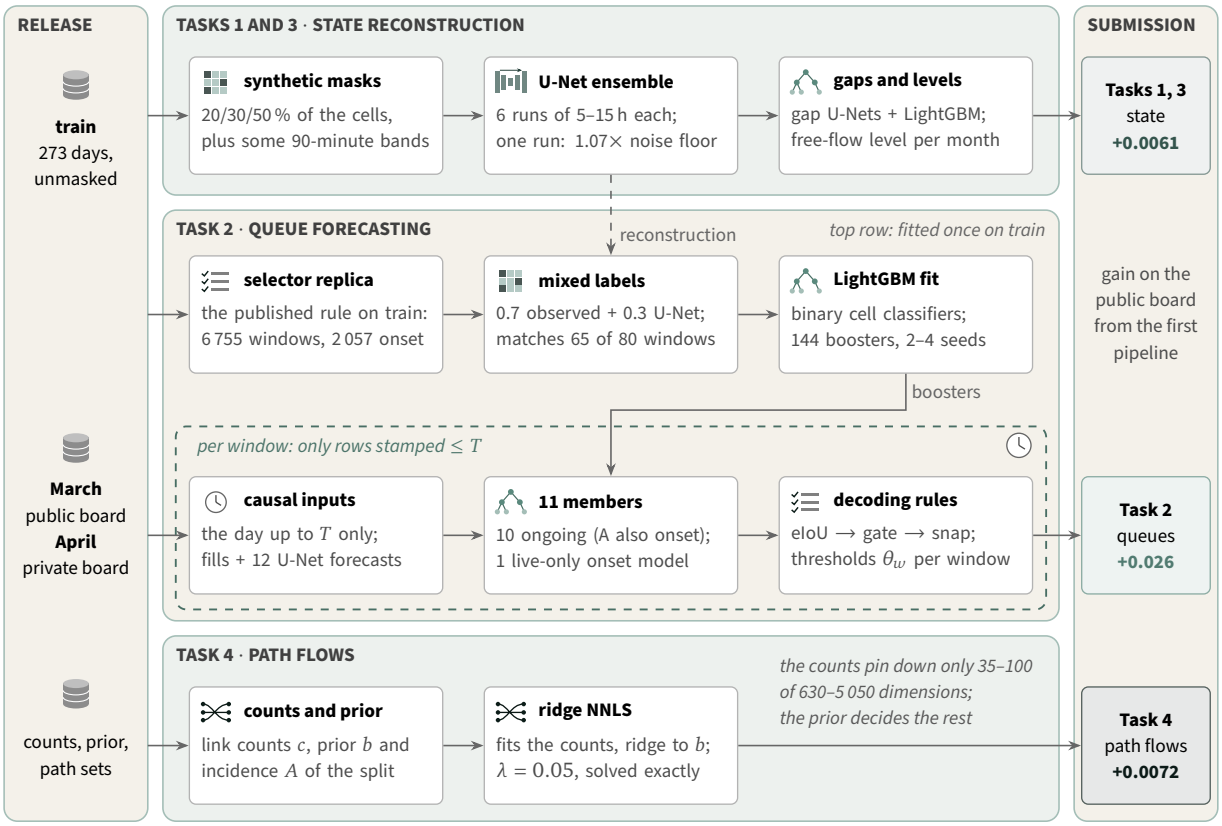


Figure 4 | **Three independent branches, one per task family.** The pipeline behind both final submissions. Three branches read the release: state reconstruction (top), queue forecasting (middle) and path flows (bottom). The queue branch is fitted once on the training months (upper row). It is then applied window by window to inputs stamped at or before the origin (dashed frame, bottom row). Right: the public-leaderboard gain of each branch from our first complete pipeline to the finals. Final B adds one rule that acts on April windows only (Section 4.3.6).

4. Method

4.1. Overview

Figure 4 shows the pipeline behind both final submissions. Three independent branches read the release. The state branch reconstructs every panel-day and serves Tasks 1 and 3. The queue branch forecasts each Task-2 window from data stamped at or before its origin. The path-flow branch solves one estimation problem per panel and split. The branches meet only in the final assembly; the queue branch uses the state reconstruction of *earlier* days only.

4.2. State reconstruction (Tasks 1 and 3)

`tfb/state/`

We treat each panel-day as an image with 288 five-minute steps along one axis and the links of the corridor along the other. A fully convolutional U-Net [17] fills the hidden cells. It has four levels (32 to 256 channels), residual blocks with group normalisation and dilated convolutions in the bottleneck. It does not predict speed and flow from scratch. It predicts a correction to a per-link linear interpolation in time, which is already a good first guess.

The network sees 25 input channels: the masked speed, flow and occupancy with their observation masks and quality fields, the interpolation baselines and gap lengths, time of day and weekday, seven static link attributes from the released diagram parameters, and the ramp counts. We train on the unmasked train layer with synthetic masks (20, 30 or 50 % of the cells, plus occasional



90-minute bands) on random crops of 64 links. The loss mirrors the score. Its first term is the Task-1 score in squared form on the hidden cells; its second term is the relative error of the vehicle-count increments, the quantity behind S_{LWR}:

$$\mathcal{L} = 1.4\left(0.54\,\frac{\mathrm{MSE}_v}{25^2} + 0.46\,\frac{\mathrm{MSE}_{q/\mathrm{lane}}}{600^2}\right) + 0.03\,\frac{\sum_t |\Delta_t e|}{\sum_t |\Delta_t N|}, \qquad e = (\hat{k} - k)\,L \text{ on hidden cells.} \tag{6}$$

The factor $1.4 = 0.35 \times 4$ carries the weight of S_{state} in Equation (1) and the slope of the RMSE at the error level of interpolation. We optimise with AdamW [14] and a cosine schedule whose length follows a wall-clock budget, and keep an exponential moving average of the weights. The submitted ensemble averages six networks trained for 5 to 15 hours each.

Long gaps after the queue windows. In the evaluation months every Task-2 horizon and the following 60 minutes are blanked on all links of a panel, and these cells remain Task-1 targets. Gaps this long and this wide are rare in the training masks. We fine-tuned two U-Nets on exactly this pattern (bands of 18 steps right after a training window's origin) and trained a LightGBM corrector per channel. It sees the network predictions, the speeds before and after the gap, the gap age, and the link attributes. It is applied in two stages: first to the six-network ensemble, then to the two fine-tuned networks, averaged half and half with the first.

Free-flow level. In free flow the generator holds each link at one speed level, plus white noise of which about 30 % is common to all links of the panel at a step (Section 6.1). The level is the same in all nine training months (between-month standard deviation 0.01–0.03 km/h), but March and April have their own: the median link moves by 0.5 to 1.1 km/h, a few by 5 km/h. A network that sees one panel-day has to guess the level from that day. We estimate it from the whole month instead, as $\ell = \mathrm{median}\{v_{\mathrm{obs}} : v_{\mathrm{obs}} > Q_{75} - 6\,\mathrm{km/h}\}$ over the link's observed speeds in the month. We then move every hidden cell that the ensemble places less than 3 km/h below this level ℓ, or above it, towards the level plus the panel's common deviation c_t. Here c_t is the mean deviation of the visible free-flow cells at that step (at least ten):

$$\hat{v} \leftarrow w\,\hat{v} + (1 - w)\,\min(\ell + c_t,\ 120.7), \qquad w = 0.15 \text{ (0.35 when } \hat{v} \text{ is 2–3 km/h below } \ell), \tag{7}$$

where 120.7 km/h is the generator's speed cap. On the held-out days the speed error of hidden free-flow cells falls from 1.196 to 1.148 km/h. Task 1 is offline, so the whole month may inform it; the Task-2 inputs do not use the correction. `tfb/state/level_apply.py`

4.3. Queue forecasting (Task 2) `tfb/queue/`

Task 2 is scored on a few deliberately chosen windows, so we model the choice itself. The queue branch has five parts: labels that approximate the organisers' noise-free truth, training windows drawn like the scored ones, inputs that never see the future, gradient-boosted cell classifiers, and decoding rules that encode what the window selection implies about the answer.

4.3.1. What counts as a queue `tfb/labels/`

The truth marks a link as queued when its *noise-free* speed is at most v_{cut}. The train split shows only noisy observations, so the noise-free state has to be estimated. Thresholding the observations mislabels cells near the cut; thresholding a U-Net reconstruction blurs the edges of queues. We use a mix of both,

$$\tilde{v} = 0.7\,v_{\mathrm{obs}} + 0.3\,\hat{v}_{\text{U-Net}} \quad \text{on observed cells,} \qquad \tilde{v} = \hat{v}_{\text{U-Net}} \quad \text{in detector gaps.} \tag{8}$$

Here $\hat{v}_{\text{U-Net}}$ comes from three denoising U-Nets trained like the state networks (Table 11). The weights were not tuned on the leaderboard. We chose them with the selector as an oracle: run the



selector replica on a candidate label field and count how many of the 80 published train windows it reproduces. Raw observations reproduce 57 windows and make 11 published windows invalid. The U-Net field gives 58 and 9, a 0.6/0.4 mix 64 and 2, and the final 0.7/0.3 mix 65 and 1.

4.3.2. Training windows and causal inputs `tfb/labels/selector.py`

Run without the monthly quota, the selector replica places 6 755 training windows on these labels (2 057 onset and 4 698 ongoing windows over nine months and eight panels): every admissible origin of each month, plus one extra pass per condition so that young queues are represented. These windows cover whole months, while the scored windows are the first of their month (Section 2.3). Section 4.3.6 returns to this mismatch.

Filled histories. The evaluation histories have detector gaps (16–18 % of the cells), and the origin row is masked like any Task-1 row. For every window we run the state U-Net on a copy of its day from which *every row after* T has been removed (observations, ramp counts and quality fields). We use the result only to fill the gaps of the last hour. Training windows are filled in the same way.

Speed forecasts. We also trained U-Nets as 30-minute forecasters: everything after a random origin is hidden, and the network predicts the speeds of the next six steps. There are four training folds and three variants: a plain one, one with an L_1 loss, and a queue-focused one with an extra queue-probability output and a stronger weight on queued cells. Training windows receive out-of-fold forecasts. An evaluation window receives the forecast of each fold model in turn, and a member averages its probabilities over the four.

4.3.3. Features and members `tfb/queue/features.py`

Each training row is one (window, link) pair for onset windows, whose answer lives at T+30 only, and one (window, link, step) triple for ongoing windows. About 100 features describe the last hour of the link and of up to three neighbours on each side: speed relative to v_{cut} and flow relative to capacity. These include occupancy and density with their means, minima and slopes, queue flags and the time since the last queued step, ramp counts, the length and the head and tail positions of the visible queue, calendar terms, and static link attributes. Three kinds of recurrence complete the picture. *Climatology* is how often the link was queued at this time of day in train; *in-month recurrence* counts the earlier days of the same month on which it was queued around this time. For some members, the U-Net speed forecasts provide the third. LightGBM [9] fits binary cell classifiers. We call each fitted LightGBM model a *booster*, and the boosters of one feature set, fitted pooled or per panel with two to four seeds, a *member*.

The onset probability blends a model with climatology half and half with a model without it; the model with climatology is itself the geometric mean of fits on our labels and on raw labels. All three are boosters of member A in Table 2. A separate *live-only* onset model sees neither climatology nor link identity nor static attributes, so it can only judge the current state. Ongoing cells use the mean of the ten members in Table 2, none of which uses climatology. Section 5.3 reports the ablations behind this mix.

4.3.4. Decoding onset windows `tfb/decode/`

An onset answer is a set of links at one time step. A probability per link is not yet an answer, so the decoding matters as much as the model. Three rules act in sequence; Appendix A gives their exact definitions.

Expected-IoU decoding. Following the decision-theoretic treatment of the IoU by Nowozin [15], we restrict the set to prefixes of the links sorted by their probability p_l and choose the length k that

Table 2 | **Ten ongoing members, averaged with equal weights.** Each feature set is fitted once pooled over all panels and once per panel. Members with a forecast input average their probabilities over the four fold forecasters ("queue" = the queue-focused variant). None of them uses climatology.

member	A	AP	B	BP	C	CP	D	DP	V2	V2P
base features (about 100)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
in-month recurrence	✓	✓	✓	✓	✓	✓	✓	✓	–	–
U-Net speed forecast	–	–	plain	plain	queue	queue	L_1	L_1	–	–
fitted per panel	–	✓	–	✓	–	✓	–	✓	–	✓
seeds	2	2	2	2	4	4	2	2	2	2

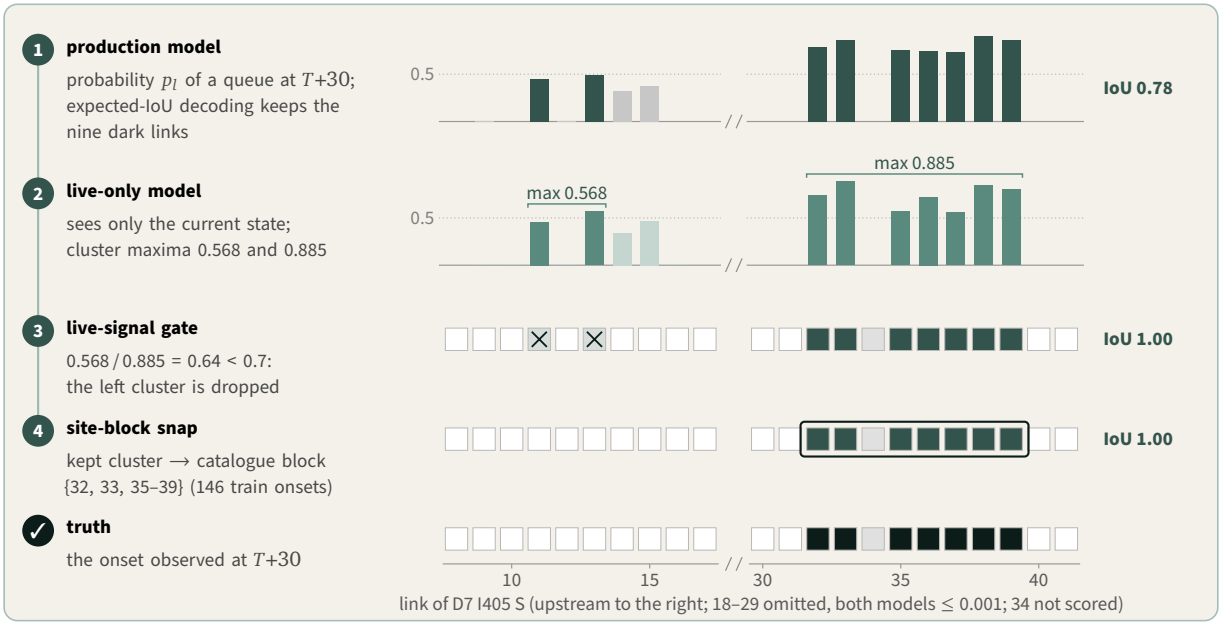


Figure 5 | **The live gate removes a train bottleneck that the current state does not support.** How one onset window is decoded (forward test: D7 I405 S, 19 January 2031; models fitted on June–November). ① Expected-IoU decoding of the production probabilities keeps two clusters, one at a bottleneck that was frequent in train. ② The live-only model, which sees only the current state, backs the right cluster much more strongly. ③ The gate drops the left cluster (ratio 0.64 below 0.7); the set now equals the truth, and the IoU rises from 0.78 to 1.00. ④ The snap confirms the kept cluster as its catalogue block.

maximises

$$\mathbb{E}\big[\mathrm{IoU}(S_k, Y)\big] = \mathbb{E}\left[\frac{|S_k \cap Y|}{|S_k \cup Y|}\right], \qquad Y_l \sim \mathrm{Bernoulli}(p_l), \tag{9}$$

by Monte-Carlo estimation with 256 samples (with 8 192 samples the decoded sets of the finals change in one March cell and no April cell). Unlike a fixed threshold, the decoder adapts the size of the set to how certain the model is.

Live-signal gate. The decoded set often holds two clusters of links: a bottleneck that was frequent in train and is backed by climatology, and a site that the current state supports. The gate keeps a cluster only if the live-only model's maximum on it reaches at least 0.7 times its maximum on the best cluster, and then re-decodes the region around the kept clusters. Figure 5 walks through one window: the gate removes a train bottleneck and leaves exactly the truth.

Site-block snap. Onset truths are not arbitrary sets of links. In every corridor they are a handful of fixed blocks, and the two most frequent blocks alone cover 69–99 % of the train onsets of a panel (Figure 6). D7 I405 S, for instance, queues as {32, 33, 35–39} in 146 train windows, and no partial variant of this block occurs more than five times. Kinematic waves [12, 16] explain part of

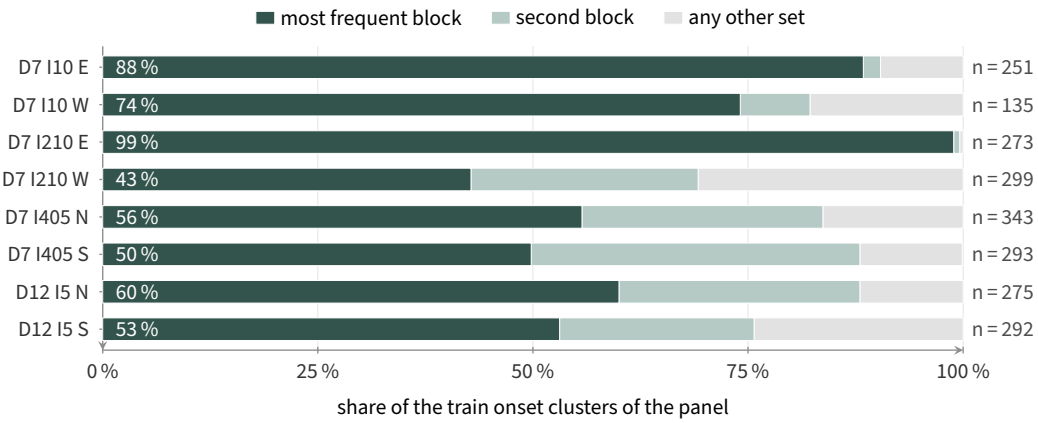


Figure 6 | **Onset truths are a few fixed link blocks.** For each panel, the share of its train onset clusters that equal the panel's most frequent block, its second block, or any other set (n: number of clusters; a window can hold two). On D7 I405 S and D12 I5 N the two blocks are two different bottlenecks; on D7 I405 N the second block is part of the first.

this: the tail of a queue moves upstream no faster than the congested wave speed of the released diagrams (19–28 km/h here). That is 1.6–2.3 km per five-minute step, so a short block fills within one step. For longer blocks the regularity is a property of the data generator rather than of the theory. Expected-IoU decoding cannot know about blocks: under independent links it keeps only the likeliest links of a block when the activation of a site is uncertain. The snap repairs this. It replaces each decoded cluster by the most frequent train block that contains the cluster's most probable link, if that block occurs at least three times and makes up at least 75 % of the blocks that contain this link.

4.3.5. Decoding ongoing windows

`tfb/decode/ongoing_ratio_thr.py`

For an ongoing window w, a cell is queued when the ten-member mean probability reaches a threshold

$$\theta_w = \kappa_w \cdot \mathrm{clip}\big(0.38\, r_w^{0.25},\ 0.25,\ 0.55\big). \tag{10}$$

The first factor follows the *month's congestion level*. Here r_w is the panel's queued share on the earlier days of the month and on the window's day up to T, relative to the same hours in train. A month that is quieter than train therefore gets a lower threshold. Queueing theory would suggest the opposite, since lower demand dissolves queues faster. We read it as a calibration shift between the months, which Section 6.5 examines. The second factor follows *why the window was chosen*: $\kappa_w = 1.5$ for spacing windows, whose queue is caught in an arbitrary phase, $\kappa_w = 1.1$ for change-point windows, and $\kappa_w = 1$ for young queues.

4.3.6. The second final submission

`run_all.sh`

Kaggle ranks the better of the two selected submissions on the private board. The second submission therefore cannot lower the final score, and a rule whose April effect is uncertain in sign belongs there. Final B equals Final A on every March cell and adds one general rule, *rolling recurrence*. For April windows, the in-month recurrence features of the eight members that use them look at the 16 days before the window's day. April's few earlier days are completed with the last days of March, which precede every April window. On the forward test this gains +0.0021 (s.e. 0.0011) ongoing IoU, and +0.0135 on windows from the start of a month. Borrowing late-February train days for March, however, lost −0.0004 on the board, so for April the rule is a hedge rather than a verified gain. It changes 58 April cells in 20 ongoing windows and no March cell. On April Final B scored

+0.00006 above Final A, against about +0.0003 that the forward gain predicts; both are below what one month of windows resolves (Section 6.6).

4.3.7. Keeping the time rule `tfb/queue/fill_causal.py`

Every Task-2 input of a window is computed from rows stamped at or before its origin: the published history and the row at T, earlier days of the same split, earlier splits, the train split, and the origins of earlier windows. The earlier days feed the in-month recurrence and the congestion ratio; the earlier origins give the selection kind. The U-Net fills and forecasts see a copy of the day without any row after T. The offline Task-1 reconstruction enters only through earlier days, and it is computed day by day. All models were fitted on the train split only. The rule also forbids later data that enters indirectly, through fitting, selection or constants written into the method. Every constant that Task 2 uses was chosen on the train split, the forward test, the March replica or the March board, all of which precede every April origin. Whole-month statistics of the evaluation months in Section 6 describe the release and informed no Task-2 choice.

4.4. Path-flow estimation (Task 4) `tfb/odme/ridge.py`

Generalised-least-squares ODME [1, 5] estimates path flows from counts by pulling the solution towards a prior. The estimator in the organisers' public Task-4 code does the same,

$$\hat{f} = \arg\min_{f \geq 0} \|A_{\mathcal{M}} f - c\|_2^2 + \lambda \, \|f - b\|_2^2, \qquad \lambda = 0.05, \tag{11}$$

and uses only the split's released counts and prior. We solve it exactly, as one non-negative least-squares problem on the stacked system $[A_{\mathcal{M}}; \sqrt{\lambda}\, I]\, f \approx [c; \sqrt{\lambda}\, b]$ [10]. The solution reproduces the released counts almost perfectly ($S_{link} = 0.99994$). Section 6.3 explains why the prior decides most of the answer.

5. Results

5.1. How the score grew

Figure 7 and Table 3 trace the path from our first complete pipeline (0.87063) to the finals (0.90985). Each step is a paired difference against the previous best submission on the same March windows; a few rows combine changes submitted together.

About +0.021 of the queue gain came from rules that encode the window selection or the structure of the truth. They include the labels chosen with the selector oracle, expected-IoU decoding, the live gate, the re-decode, the snap, and the selection-kind thresholds. About +0.005 came from the traffic models themselves, namely the members, the fills, and the calibration of the thresholds. Eight of the twenty-one steps rest on the leaderboard alone, worth +0.0082 together, and three bundled steps contain a threshold chosen there. This is the part of our public score most exposed to the winner's curse. April bore this out (Section 6.6). The same 21 steps gained +0.0253 there. The eight steps chosen on the board alone gained +0.0031, all but +0.0001 of it from the ODME ridge of row 4, against +0.0082 on March; the nine steps with local support gained +0.0143, each of them positive.

Baselines. The organisers report a persistence baseline of $S_{\text{queue}} = 0.30$ on March: onset windows score zero under persistence, ongoing windows 0.60. The calendar climatology with persistence we started from scored about 0.61, our first LightGBM members about 0.73, and the final pipeline 0.884. It scored 0.861 on onset and 0.907 on ongoing windows, measured with two block ablations of Final A that empty all onset or all ongoing windows. On April the same two ablations give 0.771 and 0.914. The Task-1 baseline is linear interpolation (Table 4).



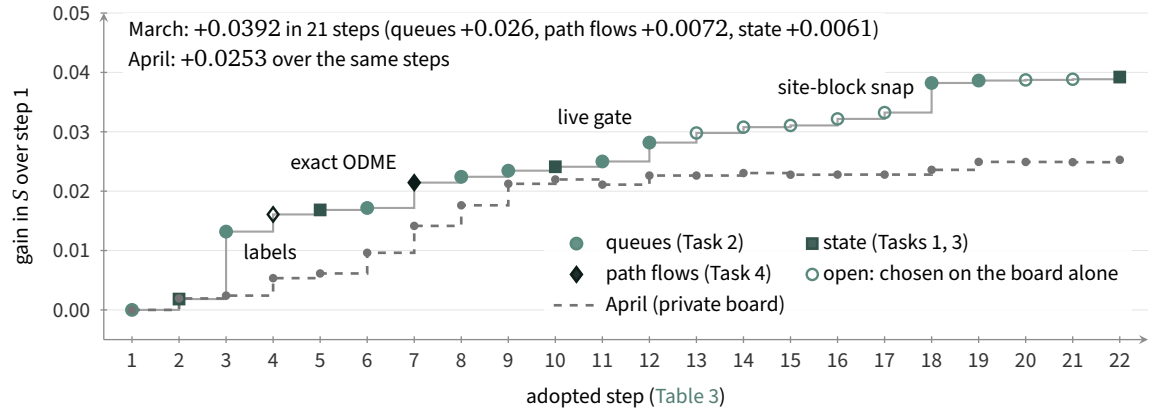


Figure 7 | **Two thirds of the gain came from the queue branch.** Gain in S over the first complete pipeline after each adopted step, on the public board (March, markers) and on the private board (April, dashed line, revealed after the deadline). Colour marks the task of the change; open markers are steps chosen on the leaderboard alone, filled markers steps with local support. The queue branch contributes two thirds of the gain, and its largest steps are the labels (bundled with three long U-Nets) and two decoding rules, the site-block snap and the live gate.

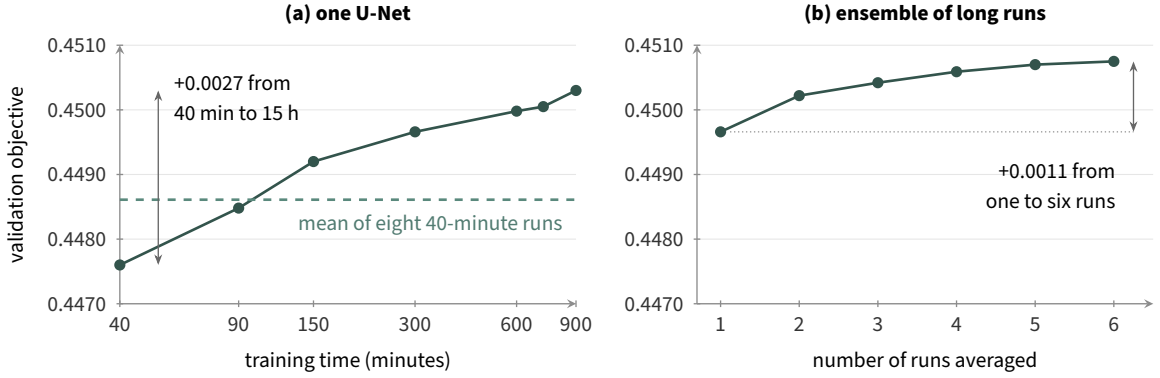


Figure 8 | **Training longer beats averaging more runs.** Validation objective $0.35\,S_{\text{state}} + 0.15\,S_{\text{phys}}$ on 31 held-out train days. (a) A single U-Net keeps improving with training time, and eight short runs averaged (dashed) do worse than one 150-minute run. (b) Averaging long runs (300 to 900 minutes) helps until about five members. Training time buys more than ensemble size.

5.2. State reconstruction

The most useful single finding on Task 1 was simple: train longer. Figure 8 shows the validation objective on the 31 held-out train days. One network gains steadily from 40 minutes to 15 hours of training. Averaging many short runs does not make up for this: eight 40-minute runs average to 0.44861, below a single 150-minute run. Averaging long runs helps, but the gain flattens after five members.

Table 4 compares the ensemble with linear interpolation. Our first state model, a gradient-boosted residual model, sat in between (about 0.93 in S_{state} and 0.72 in S_{phys}). The gain of the ensemble over it transferred one-to-one to the leaderboard: +0.0164 locally, +0.0163 on March and +0.0172 on April. The gap corrector added +0.00066 on the board, about 40 % of the gain we had simulated on held-out days (44 % on April); the simulated gaps sit at different congestion moments from the real ones. The free-flow level correction added +0.00037 on March and +0.00040 on April, against +0.00039 measured on the held-out days.

Figure 9 shows what this looks like on one held-out day with half of its eligible cells hidden. The ensemble restores every jam from the visible half, and what remains of the error looks like the



Table 3 | **Twenty-one adopted steps, measured on both boards.** Each row is a paired difference against the previous row on the same windows: ΔS_{Mar} on the public board, on which we chose the steps, and ΔS_{Apr} on the private board, revealed after the deadline; the bars show March (dark) above April (light) on one scale. Rows 3, 5, 8 and 11 combine changes submitted together. Evidence: L supported by a local test (held-out days, forward test or selector oracle), M predicted by the Task-4 score model before submission, B chosen on the leaderboard alone. The last row is Final A; Figure 7 shows the gain after each step.

	change	task	ev.	ΔS_{Mar}	ΔS_{Apr}	
1	U-Net state ensemble and LightGBM queue members	all	L	—	—	
2	longer U-Net training (150 and 300 minutes)	state	L	+0.00183	+0.00193	
3	mixed labels and three long U-Nets	queues state	L	+0.01136	+0.00048	
4	ODME ridge with the split's own counts and prior ($\lambda = 5$)	ODME	B	+0.00289	+0.00294	
5	five long U-Nets; ongoing threshold 0.5	state queues	L B	+0.00076	+0.00079	
6	causal history fills	queues	L	+0.00033	+0.00348	
7	exact solution of the published estimator ($\lambda = 0.05$)	ODME	M	+0.00427	+0.00453	
8	expected-IoU onset decoding; six long U-Nets; threshold 0.45	queues state	L B	+0.00098	+0.00347	
9	three ongoing members without climatology	queues	L	+0.00103	+0.00362	
10	gap corrector (two stages)	state	L	+0.00066	+0.00073	
11	blend at 0.40 with five members without climatology	queues	L B	+0.00089	−0.00087	
12	live-signal gate (ratio 0.5)	queues	L	+0.00318	+0.00153	
13	gate ratio 0.7	queues	B	+0.00162	±0	
14	half of the ongoing members fitted per panel	queues	B	+0.00098	+0.00042	
15	blend threshold 0.38	queues	B	+0.00029	−0.00028	
16	threshold scaled with the month's congestion ($r^{0.25}$)	queues	B	+0.00110	+0.00002	
17	onset re-decode inside the kept clusters	queues	B	+0.00107	±0	
18	site-block snap	queues	L	+0.00498	+0.00080	
19	spacing windows: threshold ×1.5	queues	L	+0.00041	+0.00134	
20	change-point windows: threshold ×1.1	queues	B	+0.00011	−0.00001	
21	queue-focused forecasters as inputs of members C and CP	queues	B	+0.00011	−0.00004	
22	free-flow level correction	state	L	+0.00037	+0.00040	

Table 4 | **The U-Net ensemble raises S_{state} by 0.045 over interpolation.** Task-1 and Task-3 scores on 31 held-out train days. R1–R3 are the three masking regimes (20, 30 and 50 % hidden).

model	S_{state} ↑	R1	R2	R3	S_{phys} ↑	objective↑
linear interpolation (baseline)	0.9096	0.915	0.911	0.903	0.6794	0.4203
gradient-boosted residual model (first model)	≈ 0.93				≈ 0.72	
U-Net ensemble (six runs)	0.9542	0.955	0.955	0.953	0.7785	0.4508
+ free-flow level correction (submitted)	**0.9551**	**0.956**	**0.956**	**0.954**	**0.7790**	**0.4512**

detector noise itself.

A late experiment used a property of the data generator we verified on the training data. The generator throttles an on-ramp to about a fifth of its usual flow, mostly to zero, in the same step in which its mainline link queues. It does not blank the ramp files inside the long gaps. Ramp-based "jam sensors" and per-link speed levels cut the corrector's speed error inside simulated gaps from 5.05 to 4.44 km/h, but on the leaderboard the effect was +0.00001. We kept the simpler corrector.



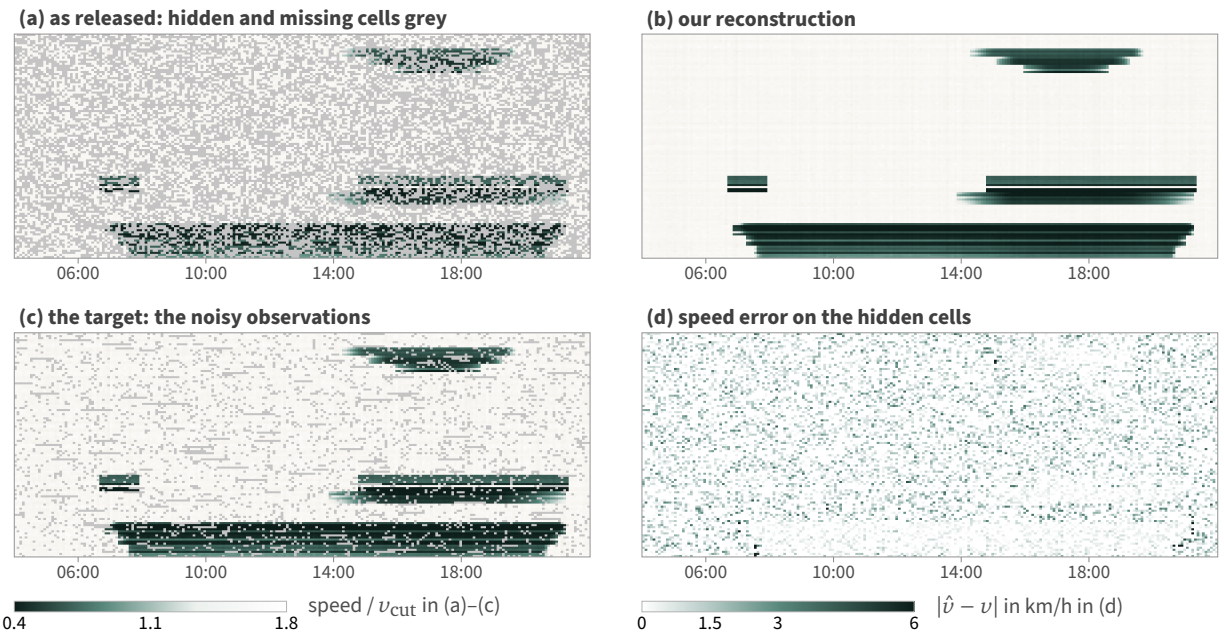


Figure 9 | **Half of the cells hidden, every jam still in place.** Task 1 on a held-out train day (D7 I405 S, 13 August 2030, regime R3, 04:00–22:00; links in driving direction from bottom to top). (a) The released layer: 38 % of the cells are hidden targets and another 16 % are detector gaps. (b) Our reconstruction: released cells as given, all others from the six-network ensemble. (c) The target, the noisy observations. (d) Absolute speed error on the hidden targets: 1.46 km/h RMSE against a detector-noise floor of 1.29 km/h on this panel. It is larger in free flow (1.56) than in and near the queues (0.93), where the generator holds a flat speed plateau, and it shows no structure: what is left is noise.

5.3. Queue forecasting

Four ablations decided the mix of models.

- **Climatology helps onset and hurts ongoing.** Where a queue will start depends on where queues usually start, so the onset model needs climatology: dropping it cost −0.0133 on the board, and relying on it alone cost −0.0093. An ongoing queue is different. It is visible, and its own dynamics say more than the training months; climatology-based ongoing members cost −0.0040.
- **In-month recurrence helps, although it is thin.** Most scored windows have only 0–3 earlier days in their month. Still, removing these features lost −0.0003 to −0.0006.
- **Half of the members are fitted per panel.** With 0, 1/6, 2/7, 1/2 and all members fitted per panel, the board scored 0.90043, 0.90066, 0.90112, 0.90141 and 0.90021.
- **The forecast members carry the blend.** Without the six members that use U-Net forecasts, the blend lost −0.0052, the largest contribution of any group of members. Better forecast metrics are no guarantee, though: training the plain forecasters longer lowered their holdout RMSE from 4.59 to 4.43 km/h but cost −0.0006.

Table 5 collects the decoding rules. Each one encodes a property of the selection rule or of the truth, and we measured each as a paired difference. The site-block snap is the clearest case of a rule that the training months under-rate. It gains +0.0029 onset IoU on the forward test, +0.025 on the March replica, and +0.0050 in S on the board, the largest single decoding gain. Changes that move away from the decoder's own set size lose: scaling the onset probabilities by 1.2 cost −0.0052, and a stacker that trimmed the sets cost −0.0108.

Table 5 | **Each decoding rule encodes a property of the selector or of the truth.** The Task-2 decoding rules, what each encodes, and its measured effect. Change in IoU of the affected condition on the forward test (standard error about 0.002) and on the March replica (Section 3), and the paired difference in S on the public (March) and the private (April) board. The gate is neutral on the training months and positive on March-like windows; the highlighted snap is positive everywhere. Board values are paired differences of submissions that changed only this rule; Table 3 bundles some of them with other changes.

rule	encodes	forward	March replica	board ΔS: March	April
labels from the selector oracle	the truth field the organisers used	65 of 80 windows reproduced		+0.0088	+0.0022
expected-IoU decoding	onset answers are sets, scored by IoU	—	—	+0.0005	+0.0031
live-signal gate	new bottlenecks differ from train ones	−0.002	+0.008	+0.0048	+0.0015
re-decode in kept clusters	the gate leaves partial sets	−0.0001	—	+0.0011	±0
site-block snap	onset truths are fixed link blocks	+0.003	+0.025	+0.0050	+0.0008
congestion-level threshold	calibration shift between months	—	—	+0.0011	±0
spacing windows ×1.5	queue caught in an arbitrary phase	—	—	+0.0004	+0.0013
change points ×1.1	queue barely moves before T+30	—	—	+0.0001	±0

We swept the kind factors on the board. For spacing windows the board agrees in direction with the forward test (×0.8: −0.00045; ×1.3: +0.00017; ×1.5: +0.00041; ×1.75: +0.00023). For change points March is flat between ×1.0 and ×1.1 (+0.00011) and falls off on either side, so this factor is a leaderboard choice of little weight.

5.4. Path-flow estimation

We had four earlier ODME submissions: the organisers' count baseline at 0.836, a blend of it with a ridge solution, ridge with $\lambda = 5$, and ridge towards the mean prior. All four scored as the official formulas predict under the estimator of Equation (11), within an RMSE of 0.002. Other values of λ fit these four scores worse (0.005 for $\lambda = 1$, 0.016 for $\lambda = 5$). A KL projection of the prior fits much worse (0.094). The exact solution raised S by +0.0043 over $\lambda = 5$, to $S_{\mathrm{ODME}} \approx 0.993$, 12 % less than the score model had predicted (+0.0049). Small values of λ are interchangeable: 0.01 and 0.05 differ by 0.03 % in L_1. The prior is not. Regularising towards the mean of the three released priors instead of the split's own one cost −0.046 (−0.23 in S_{ODME}). A refit on mean detector flows instead of the released counts moves the path flows by 15–33 % (L_1) away from the reference the four scores confirm. We did not submit it: the score model put its risk at −0.014 to −0.028, against a gain of at most +0.0014.

5.5. What did not help

Table 6 lists the larger rejections. Two groups stand out. Changes that make onset sets larger or smaller than the expected-IoU decoder proposes lose, and so do changes that lean on the locations of the training months. We also tried to couple the boosters more tightly with neural networks and with retrieval; none of it reached a final submission.

- *Network outputs as features.* Given to the boosters, the queue-focused forecaster's own queue

probability improved the forward blend by +0.0007 to +0.0010 IoU, at most +0.0002 in S, but did not help the onset model (−0.004). Mixed into the blend directly, it lost on the board (Table 6).

- *A network member.* An MLP ensemble on the same features scored 0.859 alone against 0.910 for a booster and improved no blend.
- *A learned refiner.* A 1-D CNN that refines a member's probability map along the corridor gained +0.004 to +0.007 when we trained and tested it on the same booster's predictions. Trained on out-of-fold predictions and applied to the blend, it lost −0.004: it had learned one booster's errors, not traffic structure.
- *Retrieval.* For each window we looked up the 20 most similar earlier corridor states (speed/v_{cut} over the last 20 minutes) and gave their realised queue maps to the boosters. This was the strongest idea in the forward test, +0.002 to +0.007 IoU per member and +0.004 for the blend. On the board four such members cost −0.0005. The forward gain turns negative when the mean match distance exceeds 0.15. With a gate at this distance, fixed on the forward test, the board effect was zero (−0.00002).

6. Analysis

6.1. Task 1 is limited by detector noise

The targets of Task 1 are noisy observations. No model can predict the noise, so the error has a floor. Three observations say our reconstruction is close to it.

The masking rate hardly matters. Hiding 2.5 times as many cells (R3 instead of R1) costs only 0.002 in S_{state} (Table 4). If the model were the bottleneck, less input would hurt more.

The floor can be measured. In free flow, the speed residuals around each link's free-flow level are white in time: their lag-1 autocorrelation is 0.00 to 0.03, and their first differences show the value −0.50 that white noise implies. Their standard deviation σ is 0.83 to 1.72 km/h depending on the panel, and it does not change with the flow. One part of this noise is shared: about 30 % of its variance (ρ = 0.29 to 0.33) is common to all links of a panel at the same step, however far apart they are. A reconstruction can learn the shared part from the neighbouring links, but not the rest, so the floor is

$$\text{RMSE}_{\text{min}} = \sigma\sqrt{1-\rho}. \tag{12}$$

On visible cells that we hide and reconstruct, the error of one network is 1.03 to 1.19 times this floor (median 1.07; Figure 10). Floor and error rank the ten panels in nearly the same order (Spearman 0.95).

More modelling did not help. Two further long runs, a per-panel fine-tuning (held-out +0.00015, board ±0), and a self-supervised residual corrector all left the error where it was. We stopped training state models after six runs; the last gain came from the generator's free-flow levels instead.

The evaluation months are noisier and shifted. The same self-supervised test on March and April (Figure 10) gives errors about 0.09 km/h higher than on held-out train days, worth about −0.0007 in S. A residual corrector trained on exactly this self-supervised task did not reduce the gap (1.248 against 1.250 km/h). Part of the gap was a learnable shift after all: each link's free-flow level moves between months, and the level correction of Section 4.2 lowers the self-supervised error by 3.8 % in March and 4.2 % in April. The rest behaves like a higher noise level.

Table 6 | **What lost on the board.** Changes rejected by the public leaderboard (paired ΔS against the best submission of the day), sorted by effect, and the same pairs on the private board (April), revealed after the deadline. The bars (March) share one scale; the first entry is cut at the scale's end. Analog features summarise the 20 most similar earlier corridor states (Section 5.5). The head probability is a U-Net's direct queue probability. Corridor-load features give the vehicles on the corridor relative to its critical load and their 30-minute change. Location-free members drop link identity and position. Whole-day in-month activity is the share of the month's earlier days on which a link queued at any time.

change	Mar.	Apr.	change	Mar.	Apr.
ODME towards the mean of all priors	−0.0457	−0.0458	live gate ratio 0.85 instead of 0.7	−0.0019	−0.0016
onset model without climatology	−0.0133	−0.0024	raising thresholds on quiet panels	−0.0016	±0
onset stacker (smaller sets)	−0.0108	−0.0047	whole-day in-month activity	−0.0013	+0.0001
onset from the climatology model only	−0.0093	−0.0092	head probability in the blend (20 %)	−0.0012	+0.0006
onset location features from earlier days	−0.0062	−0.0015	twelve instead of ten members	−0.0009	+0.0001
onset probabilities ×1.2 (larger sets)	−0.0052	−0.0002	location-free ongoing members	−0.0009	+0.0007
climatology-based ongoing members	−0.0040	+0.0004	per-window loss weights	−0.0009	+0.0001
in-month features capped to the month start	−0.0032	−0.0017	observation-first labels	−0.0009	−0.0010
analog features in all ten members	−0.0027	−0.0027	forecast members weighted ×1.5	−0.0007	+0.0001
forecast members only	−0.0027	+0.0003	plain forecasters trained longer	−0.0006	−0.0003
			corridor-load features	−0.0006	+0.0002
			analog features in four members	−0.0005	−0.0004
			selection features in the ongoing members	−0.0005	+0.0012
			rolling recurrence from February	−0.0004	+0.0010

6.2. Task 3 is limited by the noise in its targets

S_{FD} is close to 1 for any plausible state (0.986 for ours). Thus S_{phys} is decided by S_{LWR} (0.67 on held-out days). The scorer compares the reconstructed vehicle-count increments with the *observed* ones, so the noise of the targets enters directly. Figure 11a puts our reconstruction next to three reference states. The targets themselves, the observations, score 0.994; the observations with a second dose of detector-level noise score only 0.715, and a three-step smoothing of them 0.765; our reconstruction reaches the smoothed observations. Figure 11b shows why the score cannot go much higher. Transitions that touch a congested cell are only 4.5 % of all transitions but carry 21 % of the conservation error, because $k = q/v$ amplifies speed errors at low speeds. The free-flow transitions carry the other 79 %, because noise dominates their small increments. Projecting the predicted cells onto the fundamental diagram raised S_{FD} by at most 0.0015 and cost S_{LWR} and S_{state}, so we submit the unprojected reconstruction.



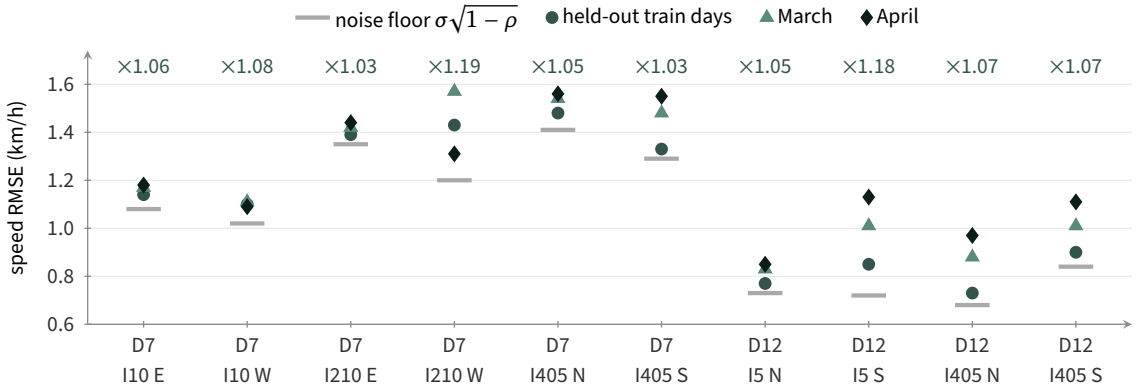


Figure 10 | **The speed error sits close to each link's detector-noise floor.** Speed error of one U-Net on hidden visible cells (5 % of the visible cells of each month hidden and reconstructed) against the link-specific noise floor of the observations. The numbers on top give the ratio of error to floor on held-out train days. March and April are slightly harder, mostly on the panels with new bottlenecks (D12 I5 S, D12 I405 N).

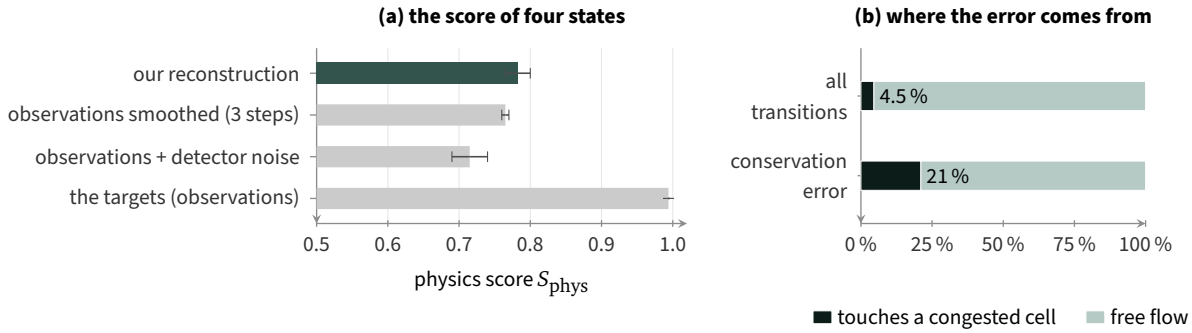


Figure 11 | **The physics score is bounded by the noise of its own targets.** (a) S_{phys} of our reconstruction and of three reference states on held-out train days (bars: mid-points; whiskers: range over panels). The noisy targets score almost 1 against themselves; any state without their exact noise lands near 0.7–0.8. (b) Congested transitions are rare but carry a fifth of the conservation error; most of it is noise in free flow.

6.3. Task 4 is decided by its prior

The counts pin down far fewer directions than there are paths. On the ten panels the counted incidence $A_{\mathcal{M}}$ has rank 35 to 100, against 630 to 5 050 paths (Figure 12). In each of the 595 to 4 950 directions of its null space, the counts say nothing and the regulariser of Equation (11) alone decides. This is why the choice of λ hardly matters while the choice of the prior matters a great deal (Section 5.4): with a null space this large, the prior *is* the answer in most directions.

6.4. Where the queue errors are

The evaluation months have no labels, so we cannot inspect a March or April window. Block ablations still give the March level per panel, and the forward test shows which kinds of error remain (Figure 13).

March, panel by panel. Onset windows are almost solved on five of the eight panels (IoU 0.90–1.00). Of the 5.6 onset window units lost in March, 4.7 fall on three panels: D12 I5 N (0.50), D12 I5 S (0.76) and D7 I10 E (0.80). Ongoing windows lose less per window but on more panels. The three weakest, D12 I5 S (0.79), D7 I210 W (0.81) and D12 I5 N (0.85), are all quieter in March than in train. Up to each window's origin their queued share is 0.36–0.89 times that of the same hours in the training months. Quietness alone does not explain the loss, though; D7 I405 N is just as quiet (0.42) and scores 0.93. These per-panel values come from block ablations of earlier

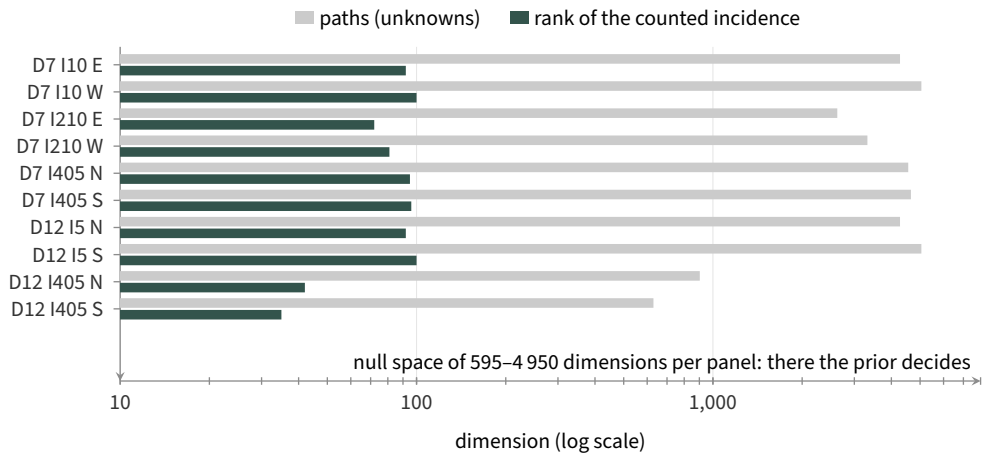


Figure 12 | **Task 4 is badly under-determined.** For every panel the link counts constrain 35–100 directions of a path-flow vector with 630–5 050 entries.

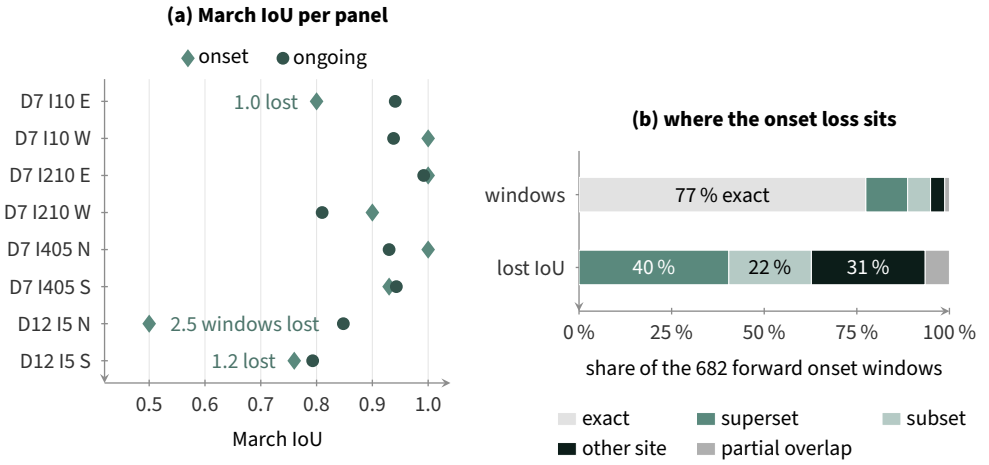


Figure 13 | **Three panels hold most of the lost onset windows.** (a) March IoU per panel from block ablations on the public leaderboard (each value from the submission that was current when the panel was measured). (b) Error classes of the forward onset windows after expected-IoU decoding, gate and re-decode, before the snap. Supersets hold an extra cluster beside the true one, subsets miss part of the true block, "other site" sets do not touch the truth, and partial overlaps share some links with it.

submissions; Final A's March levels are 0.861 for onset and 0.907 for ongoing windows. On April they are 0.771 and 0.914. The panel ablations emptied March windows only, so April has no values per panel.

Onset errors. On the forward test, 77 % of the 682 onset windows are decoded exactly. The remaining loss has four sources. *Supersets* (11 % of the windows, 40 % of the loss) contain the true block and an extra cluster beside it, usually a bottleneck that was frequent in train. *Subsets* (6 % of the windows, 22 % of the loss) hold part of the true block; the snap repairs most of them. *Other sites* (4 % of the windows, 31 % of the loss) miss the truth entirely, and *partial overlaps* (1 % of the windows, 7 % of the loss) share some links with it without containing it or lying inside it. 46 of the 682 windows (6.7 %) have truths outside the corridor's block catalogue, and at this stage the pipeline scores 0.35 on them against 0.92 elsewhere. This last class looks like a limit of the data rather than of the model. An independent model family agrees: the queue-focused U-Net forecaster gives the true site a median probability of 0.001 in these windows. Some onsets are simply not visible an hour ahead; the release adds five incidents to March and seven to April at random times

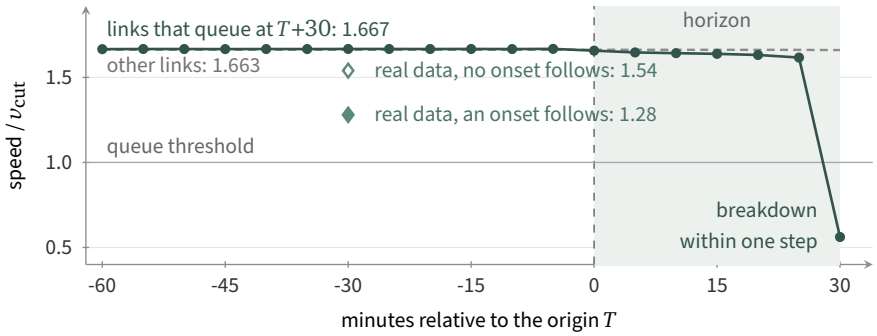


Figure 14 | **Onsets give no warning before the origin.** Speed around the onset in the 2 057 train onset windows: links that queue at T+30 against the other links of the same windows. Before the origin the two cannot be told apart, unlike the real detector data quoted in the release configuration (diamonds); the slowdown starts at T and the breakdown takes one step.

and places [18].

Onsets give no warning before the origin. The release configuration motivates the onset condition with real detector data: an hour before a queue forms, the road runs at 1.28 times its queue threshold, against 1.54 times where none follows [18]. The synthetic release does not show this. Over the 2 057 train onset windows, the links that queue at T+30 run at 1.667 times v_{cut} from T−60 to T−5, like the other links of the same windows (1.663). Their speed starts to fall only at the origin and is still at 1.617 at T+25 before the breakdown (Figure 14). Measured against the link's free-flow level of the month and the panel's common deviation, the deficit before T is 0.0 km/h. Which site breaks down therefore depends on the time of day and on the flow pattern at T, not on an early slowdown, and the models already read both. In the 296 forward windows with two or more frequent blocks, the block with the highest probability is the true one in 96 % (production model) and 97 % (live-only model). The block's onset frequency at that time of day picks it in 86 %. The flow at T relative to the block's own typical breakdown flow picks it in 60 %, and the flow relative to the nominal capacity in 31 %, below the 38 % of a random pick. The onset loss lies in the extent of the set and in sites outside the catalogue, not in the choice between known sites.

Ongoing errors. For ongoing windows we use the six-member forward blend (members A, AP, B, C, D and V2 of Table 2, fitted on June–November) at the base threshold 0.38. We test it on the 611 windows (372 change-point, 236 spacing and 3 young ones) that the selector replica places on one timeline like the scored ones (Figure 15). The IoU falls with lead time, and the selection kind matters more than the lead time (the figure pools the cells of each step; the scores below are window means). Change-point windows score 0.972 on average, because their queue barely moves before T+30. Spacing windows score 0.910: their queue is caught in an arbitrary phase, and they carry 5.8 false-positive and 2.7 false-negative cells per window, against 3.3 and 1.8 for change points. The gap between the two kinds widens from 0.024 at T+5 to 0.042 at T+30. This is the reason for the kind-specific factors in Equation (10). In member A's forward forecasts, two thirds of the loss lie within three links of the queue present at the origin, in its extent, and one third in new queues elsewhere, which the forecasts mostly miss. The forecasts already respect the selector's own condition on ongoing windows, a persistence IoU of at most 0.9. Of member A's forward forecasts for 1 535 such windows, only 2.3 % come within 0.95 of persistence, and a decoder that enforced the condition would gain about 10^{-4} in S.

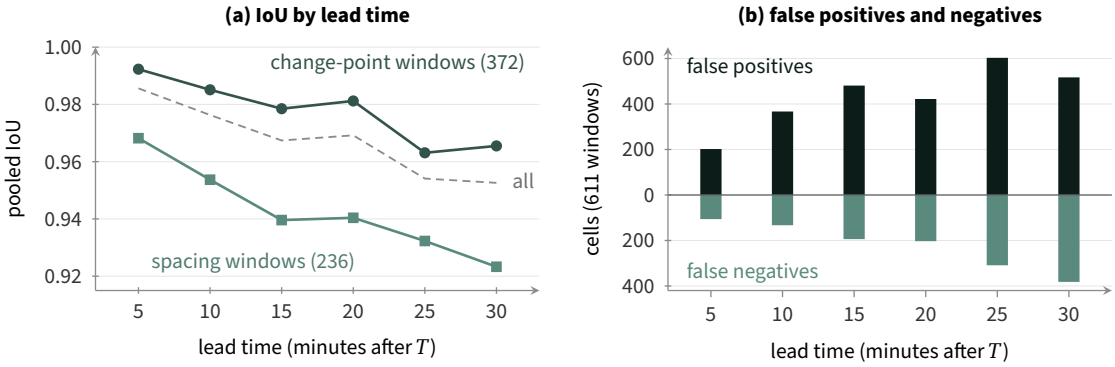


Figure 15 | **Errors grow with lead time, and spacing windows are harder at every step.** Ongoing errors on the forward test by lead time (six-member blend at the base threshold 0.38; 611 windows placed like the scored ones). (a) Pooled IoU per step for change-point and spacing windows; spacing windows are harder at every step. (b) False positives (above the axis) outnumber false negatives (below) two-to-one at this threshold.

6.5. Calibration differs between months

At 0.38 the blend predicts too many cells on the training months: there are twice as many false positives as false negatives (2 592 against 1 327). For calibrated probabilities the IoU-optimal threshold is about $J/(1+J)$ for an attainable IoU J, the Jaccard form of the F1 result of Lipton et al. [13]. At a forward IoU of about 0.92 that is 0.48, and the forward optimum of the blend is indeed 0.46–0.50 (Figure 16). March, however, rewarded every lowering of the threshold, from 0.5 to 0.45, 0.40 and 0.38. In March the models' probabilities are too low for the queued cells, and a lower threshold compensates. The congestion-ratio rule of Equation (10) carries the same shift per panel: quiet panels, where the probabilities are lowest, get the lowest thresholds. Why the probabilities are too low is open. One reading fits our observations. The scored windows sit at the start of the month, where the in-month recurrence is nearly empty, and in a quiet month the neighbouring links are queued less often as well, so the boosters see little support for a queue that the selection rule guarantees. On the March replica the congestion-ratio rule changes member A's ongoing IoU by +0.0031 (s.e. 0.0054) on the 38 windows from the first week of the month and by −0.0014 (s.e. 0.0008) on the 415 later ones, the signs this reading predicts. Neither difference reaches two standard errors, so the reading stays a hypothesis. April, whose scored windows also sit at the start of the month, does not support it. There the congestion-ratio rule gained +0.00002 against +0.0011 on March, and neither later lowering of the threshold gained (0.45 to 0.40: −0.0002; 0.40 to 0.38: −0.0003). These differences lie below what one month resolves (Section 6.6), but they suggest that the low threshold was a property of March rather than of the start of a month.

6.6. What transfers to a new month

Local tests and the leaderboard often disagreed (Table 7). The state model transferred one-to-one. Queue gains transferred in part. Gains that rest on regularities of the training months did not transfer at all. Per-window loss weights, corridor-load features, analog features, a rolling recurrence that borrows late-February days for early March, and an onset stacker all won locally and lost on March. On April the large loss of the onset stacker repeated (−0.031 IoU), while the six smaller losses of Table 7 scattered around zero, five of them positive, all within what one month resolves (end of this section).

The pattern has a simple reading. The organisers generated each evaluation month as an independent draw with its own incidents. The traffic physics stays the same: in March and April the 99th-percentile flows and the flows of queued cells lie within 2 % of the training months on

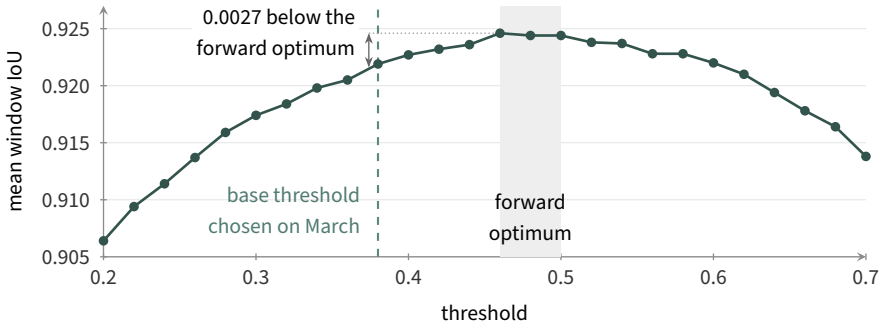


Figure 16 | **March wants a lower threshold than the training months.** Window IoU of the six-member forward blend (fit June–November, test December–February) against the decision threshold. The training months prefer 0.46–0.50, close to the calibrated optimum $J/(1+J) \approx 0.48$; the March leaderboard rewarded lower thresholds down to 0.38, which costs 0.0027 on the training months.

Table 7 | **The state model and the structural rules transferred; train-month patterns did not.** Local effect against the leaderboard effect of selected changes on March and on April (positive = better); the last column judges the transfer to March, as we saw it before the deadline. Unit: S = total score, IoU = mean IoU of the affected condition, share = part of the simulated gain that reached the board, RMSE$/S$ = local speed error on the blanked spans against the board's ΔS.

change	unit	local	March	April	transfers
U-Net state instead of gradient boosting	S	+0.0164	+0.0163	+0.0172	✓
ramp-sensor features in the gap corrector	RMSE, S	−12 % RMSE	+0.00001	+0.00003	≈
gap corrector	share	100 %	about 40 %	44 %	✓
first queue models, onset	IoU	+0.31	+0.109	+0.130	✓
first queue models, ongoing	IoU	+0.21	+0.120	+0.119	✓
site-block snap	IoU (onset)	+0.003	+0.033	+0.005	✓
onset stacker	IoU (onset)	+0.005	−0.072	−0.031	✗
climatology-based ongoing members	IoU (ongoing)	+0.018	−0.027	+0.003	✗
per-window loss weights	IoU (ongoing)	+0.0016	−0.006	+0.001	✗
head probability mixed into the blend (10 %)	IoU (ongoing)	+0.0009	−0.004	+0.002	✗
analog features (four members)	IoU (ongoing)	+0.004	−0.003	−0.003	✗
selection features (ten members)	IoU (ongoing)	+0.0036	−0.003	+0.008	✗
rolling recurrence from the previous month	IoU (ongoing)	+0.0021	−0.002	+0.007	✗

every panel. What changes is where and how often queues form, and the free-flow speed level of each link. Features that encode *where queues usually are* help when a month resembles the training months and hurt where it has new bottlenecks. Features that read the current month transfer: the live state, the in-month recurrence and the month's congestion level. The structural decoders transferred best, because they encode properties of the selector and of the data generator rather than of particular months. As booster features the same selector properties did not transfer. Seven window-level features (gap to the previous window, kind, change of the queued link set in the last ten minutes) gained +0.0036 IoU in the forward test. On March they cost 0.00045 in S, within the sampling error of 40 windows but no gain.

The private month. April was scored but unseen until the deadline. Its windows lie on days 1–17, so for most of them the earlier April days and the whole of March are history stamped before the



origin, which the rules allow. By the congestion ratio, April is quieter than the training months on seven of the eight panels (0.47–0.99; D7 I210 E 1.09), so we expected the threshold rule to matter there as well. Final B extends the in-month recurrence of April windows into the end of March (Section 4.3.6). Because Kaggle keeps the better of the two finals, a rule that the March board cannot test costs nothing in Final B; we measured every rule of Final A on the March board.

The private month, revisited. After the deadline Kaggle revealed the April score of every submission, so each paired difference in this report has an April counterpart (Tables 3 and 5 to 7). Final A scored 0.89715 and Final B 0.89721, 0.0127 below March. Nearly all of the drop sits in the onset windows, whose IoU fell from 0.861 to 0.771 (−0.0135 in S); the ongoing windows rose from 0.907 to 0.914, and the other tasks moved by −0.0003. An earlier pipeline lost 0.072 on onset as well (0.792 to 0.720): most of the drop belongs to April's windows, the rest to rules that gained less there. The paired differences held where they were large. Of the 67 distinct pairs with a March effect of at least 0.0003, the 20 of at least 0.002 kept their sign in 18 cases, the 47 smaller ones in 24. The April values of the smaller ones scatter by 0.0012 (standard deviation) around the March values, the sampling error that Section 3.2 expected, so one month of windows resolves about 0.002 in S. The winner's curse followed. The eight steps chosen on the board alone gained +0.0031 on April against +0.0082 on March, and only +0.0001 without the ODME ridge; the nine steps with local support all gained again, +0.0143 against +0.0242. The congestion-ratio rule was neutral (+0.00002, Section 6.5), Final B's rolling recurrence added +0.00006, and both CPU rebuilds scored within 0.00002 of the selected files (Section 7).

7. Reproducibility

The award rules ask for code that rebuilds the selected submissions from the released data within 1 % of their private score. They also require a Task-2 pipeline whose forecasts do not change when every row after a window's origin is removed. This section shows how we checked both.

7.1. The package

The package (Table 8) holds the code, a model archive, and the two submitted files. A single entry script, `run_all.sh`, turns the release into both final submissions. It trains nothing: every network and booster is loaded from the archive. The U-Nets were trained with wall-clock budgets on an Apple M2 Pro (32 GB, PyTorch MPS backend), so a retrained ensemble would differ slightly. The training commands, seeds, and schedules are documented for anyone who wants to retrain. Figure 17 shows where the time goes on a CPU machine without a GPU.

The pipeline runs on Python 3.12 with NumPy 2.5.3, pandas 3.0.6, PyArrow 25.0.1, SciPy 1.18.1, LightGBM 4.7.0 (with narwhals 2.27.1) and PyTorch 2.14.0, pinned in `requirements.txt`. With the release and the array cache it needs about 40 GB of disk. Table 11 and `docs/TRAINING.md` list the seeds and schedules of all networks, and the README gives the sha256 of both rebuilt files. The code is released under the MIT licence. Four file readers in `tfb/odme/baseline_io.py` re-implement those of the organisers' public baseline repository [18], which is also MIT-licensed. The release is synthetic and contains no personal data; we use it under the competition rules, and the model archive holds only our trained weights and labels derived from the train split.

7.2. Clean-room rerun

We cloned the package from its repository into an empty directory on a second machine: Linux on an AMD Ryzen 5 PRO 4650G, 8 of its 12 hardware threads available, 22 GB RAM, no GPU. There we unpacked the model archive after checking its checksum, ran the script, and compared every row with the files selected on Kaggle (Table 9). The run took 77 minutes; the final package, with



Table 8 | **Contents of the code package.** We use no external data and no pretrained weights; we fitted every model on the train split only.

part	content
`run_all.sh`	single entry script: release and models → `submission_A.csv`, `submission_B.csv`
`tfb/`	the code, one sub-package per stage: `cache`, `state` (Tasks 1 and 3), `labels`, `queue` and `decode` (Task 2), `odme` (Task 4), and the assembly
`tools/`	the Task-2 time-rule test and a block-by-block comparison with the submitted files
`docs/TRAINING.md`	every training command with seeds, schedules, and shipped epochs
model archive (2.0 GB)	20 U-Net checkpoints, 148 LightGBM boosters (144 for the queue members, 4 for the gap corrector), the train-derived queue labels and training windows
release assets	the two files exactly as submitted on Kaggle

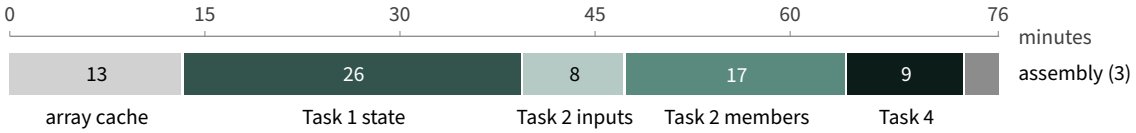


Figure 17 | **Both finals rebuild in 76 minutes on a CPU.** Runtime of `run_all.sh` by stage in the clean-room run of the final package (1 November 2026) on 8 hardware threads of an AMD Ryzen 5 PRO 4650G, Ubuntu 24.04, 22 GB RAM, no GPU; 76 minutes in total. The pipeline's processes peak at 12.1 GB of resident memory. With a GPU, the U-Net stages shrink; the result is the same up to floating-point noise (Table 9).

the free-flow level stage, rebuilt in the same checkout in 76 minutes. The speeds of the state rows move by about 10^{-4} km/h RMS (23 of 6.7 million rows by more than 10^{-3}, at most 0.29 km/h), the flows by $8.5 \cdot 10^{-4}$ veh/h RMS, and the path flows are identical.

Package versus submitted files. The package differs from the code that built the submitted files in one respect: after submission, each window's Monte-Carlo decode received its own random stream, because the time-rule test showed that a shared stream let a forecast depend on which other windows existed (Section 7.3). Together with floating-point flips between the Apple GPU that built the submitted files and a CPU, a rebuild differs in 4 (Final A) and 11 (Final B) of the 87 000 April queue cells, which can move the private score by at most 0.00015 and 0.00032, far inside the 1 % rule. The bound holds because one window weighs 0.00375 in S, and changing k cells of a predicted set P moves its IoU by at most $k/|P \cup P'|$. The seed change itself moves three onset cells in two March windows and no April cell; the flips, at LightGBM split thresholds that are met almost exactly, account for six March cells and all April cells. On the public board the rebuilt Final A scored 0.90517 against 0.90985 for the selected file, and the rebuilt Final B the same, since Final B changes only April cells. About 0.0044 of the gap comes from the two March windows, in which two candidate sets have nearly the same expected IoU and the old stream had picked the one that the board rewards. On the private board, which the 1 % rule refers to, the rebuilt Final A scored 0.89713 against 0.89715 and the rebuilt Final B 0.89723 against 0.89721, both within 0.002 % of the selected files.

7.3. The time rule, tested

The organisers check Task 2 by rerunning a team's code with every row stamped after a window's origin removed. We ran the same test ourselves. `tools/causality_test.sh` writes a copy of the release in which it deletes every row of one month after a cut time: day files, ramp files, window histories, and the window index. It reruns the Task-2 part of the pipeline on the copy and compares the forecasts of all windows whose origin lies at or before the cut with those of the full run. Windows with a later origin keep their full history in the organisers' check, so they are not

Table 9 | **The CPU rebuild matches the selected files up to a few cells.** Clean-room rerun on a Linux CPU machine against the submissions selected on Kaggle. Queue cells differ for two reasons: floating-point flips at LightGBM split thresholds (6 March cells and all April cells) and the per-window decode seed introduced after submission, 3 March cells (Section 7.3).

block	rows	Final A	Final B
state rows (Tasks 1 and 3)	6 735 795	speed: 23 rows above 10^{-3} km/h (max 0.29)	same as A
		flow: RMS $8.5 \cdot 10^{-4}$ veh/h (max 1.7)	same as A
queue cells, March	87 000	9 differ	9 differ
queue cells, April	87 000	4 differ	11 differ
path flows	70 708	identical	identical

Table 10 | **No forecast changes when the release ends at a window's origin.** Windows with origin at or before the cut, rerun on the truncated release, against the full run. *Later day files deleted instead of emptied.

cut (window origin)	March windows	April windows	cells compared	differing cells (A / B)
1 March, 09:10	4	—	3 522	0 / 0
1 March, 10:05	5	—	4 398	0 / 0
1 March, 14:15	10	—	9 786	0 / 0
2 March, 17:15	28	—	29 466	0 / 0
1 April, 19:05	80	11	101 382	0 / 0
2 April, 07:30*	80	15	106 362	0 / 0
3 April, 09:20	80	31	124 128	0 / 0
5 April, 16:05	80	61	157 416	0 / 0
7 April, 14:00	80	71	167 292	0 / 0
17 April, 15:15	80	80	174 000	0 / 0
all ten cuts			877 752	0 / 0

compared. `TASK2_ONLY=1 bash run_all.sh` runs only what Task 2 needs, which is how the test reruns the pipeline (about 45 minutes per cut on the clean-room machine). Table 10 lists the ten cuts, each at the origin of one window; the last one compares all 160 windows. No cell changes at any cut, for either final. Each cut tests one window at its own origin and every earlier window only against the rows after the cut. Since all windows run through the same code, a dependence on later rows would show in the ten windows tested at their origins.

The test found one real problem, though not a leak. The expected-IoU decoder estimates its expectation by Monte-Carlo sampling, and it first drew the samples of all windows from one random stream. In a truncated release with fewer windows the stream shifted, and at one March cut eleven cells of a single window changed. Nothing stamped after the origin had been used, but the forecasts still depended on which other windows existed. Each window now draws from its own stream, seeded by its identifier; Section 7.2 gives the effect on the submitted files.

Why the check needs a rerun. The release blanks the 90 minutes after each origin in the Task-1 files, but the blank does not hide what happens in the horizon. In the train replica's onset windows, on average 83 % of the links that first queue at T+30 are still queued at T+95, the first step after the blank. A forecast that read those rows would therefore mostly know the onset site. The on-ramp counts, which are not blanked, say much less: at T+30 their jam response flags 2.5 % of the onset cells that have an on-ramp. Only a rerun on truncated data separates a forecast from a reconstruction.

8. Limitations and conclusion

TrafficFlowBench rewards two different skills. The offline tasks reward patience. A single U-Net trained for many hours comes within 3–19 % of the detector-noise floor of the speeds. Each link's free-flow level in the month closes part of the rest, the physics score is bounded by the same noise, and Task 4 is decided by its own published objective. Beyond these points, a better model would have to predict the noise.

The online task rewards reading the benchmark. Eighty windows a month decide the queue score, and a documented rule places them. Most of our gain came from taking that rule seriously: labels and training windows that reproduce the selector, a decoder that knows an onset queue starts at the last step, a live-signal gate that tells new bottlenecks from old ones, a snap to the corridor's onset blocks, and thresholds that follow the reason a window was chosen. The generic ideas fared worse. A learned refiner looked good until we tested it on out-of-fold predictions. A network member did not reach the boosters, and retrieval of similar corridor states, the strongest idea on the training months, lost on March.

The second lesson is about measurement. One change per submission, block ablations for levels, and a forward test fitted on the past and scored on the future told us which local gains were real. They also told us where a rule belongs: one that the March board cannot test belongs, at most, in the second final. The private board drew the line for this benchmark: paired differences of 0.002 or more carried over to April, smaller ones did not, so a step that rests on the board alone should be at least that large.

Limitations. Our evidence for April was indirect until the deadline. Paired differences are exact for March but carry the sampling error of the per-window differences as estimates for April; the revealed scores put that error at about 0.0012, and the small steps that rested on the March board alone added almost nothing on April. The structural rules are tied to this benchmark: they exploit a published selection procedure and regularities of a synthetic data generator, and would need re-deriving for real detector data. Task 4 measures agreement with the organisers' reference estimator rather than true path flows, and the board does not reveal why our exact solution scores 0.993, not 1. Two error classes remain: onsets at sites the training months never showed (6.7 % of the forward onset windows, which score 0.35), and the extent of moving queues, which holds two thirds of the ongoing loss. Both look like the limit of what an hour of history can tell.

A suggestion for the benchmark. For a future edition we would suggest drawing the scored origins at random from all admissible ones instead of taking the first that qualifies. The windows would then spread over the month, an onset could appear at any horizon step, and the queue score would reward forecasting more than knowledge of the selection. The 90-minute blank after each window deserves a second look as well: 83 % of the links that first queue at T+30 are still queued at T+95 (Section 7.3), so the blank alone does not keep the horizon hidden.

References

[1] Michael G. H. Bell. The estimation of origin-destination matrices by constrained generalised least squares. *Transportation Research Part B: Methodological*, 25(1):13–22, 1991.

[2] Avrim Blum and Moritz Hardt. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1006–1014, 2015.

[3] Werner Brilon, Justin Geistefeldt, and Matthias Regler. Reliability of freeway traffic flow: A stochastic concept of capacity. In *Proceedings of the 16th International Symposium on Transportation and Traffic Theory (ISTTT)*, 2005.

[4] Wei Cao, Dong Wang, Jian Li, Hao Zhou, Lei Li, and Yitan Li. BRITS: Bidirectional recurrent imputation for time series. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, 2018.

[5] Ennio Cascetta. Estimation of trip matrices from traffic counts and survey data: A generalized least squares estimator. *Transportation Research Part B: Methodological*, 18(4–5):289–299, 1984.

[6] Andrea Cini, Ivan Marisca, and Cesare Alippi. Filling the G_ap_s: Multivariate time series imputation by graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

[7] Carlos F. Daganzo. The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory. *Transportation Research Part B: Methodological*, 28(4):269–287, 1994.

[8] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.

[9] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.

[10] Charles L. Lawson and Richard J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, 1974.

[11] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018.

[12] M. J. Lighthill and G. B. Whitham. On kinematic waves II. A theory of traffic flow on long crowded roads. *Proceedings of the Royal Society of London. Series A*, 229(1178):317–345, 1955.

[13] Zachary C. Lipton, Charles Elkan, and Balakrishnan Narayanaswamy. Optimal thresholding of classifiers to maximize F1 measure. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*. Springer, 2014.

[14] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[15] Sebastian Nowozin. Optimal decisions from probabilistic models: The intersection-over-union case. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 548–555, 2014.

[16] Paul I. Richards. Shock waves on the highway. *Operations Research*, 4(1):42–51, 1956.

[17] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[18] TrafficFlowBench organizers. TrafficFlowBench: 2026 IEEE Big Data cup – data release, evaluators and baselines. `https://github.com/jacky850/trafficflowbench-public` and the Kaggle competition pages, 2026.

[19] TrafficFlowBench organizers. Clarifications on the Task-2 time rule, the window-selection procedure, the use of earlier months and the reproducibility check. Kaggle competition discussion, topics 742068, 745700, 748789, 749278 and 749601, 2026.

[20] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. Graph WaveNet for deep spatial-temporal graph modeling. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.



A. Decoding rules

The rules turn probabilities into the submitted sets. They act on three inputs: the onset production model's probability p_l that link l is queued at T+30, the live-only model's probability q_l for the same cell, and the ten-member mean probability $\bar{p}_{h,l}$ of an ongoing window at step T+5h. A *cluster* of a link set is a maximal run of its links in corridor order where neighbouring links are at most two positions apart. Every rule uses only data stamped at or before the window's origin T.

Rule 1: expected-IoU decoding. Sort the links with $p_l > 0.003$ by decreasing p_l and let S_k hold the first k of them. With independent $Y_l \sim \text{Bernoulli}(p_l)$, decode $S = S_{k^*}$ with

$$k^* = \arg\max_{1 \le k \le 40} \widehat{\mathbb{E}}\left[\frac{|S_k \cap Y|}{|S_k \cup Y|}\right], \tag{13}$$

estimated from 256 Monte-Carlo samples of Y. The samples of a window come from its own random stream, seeded with the CRC-32 of the window identifier.

Rule 2: live-signal gate and re-decode. Let $c_1, \ldots, c_m$ be the clusters of S. If $m \ge 2$, keep cluster c_j when

$$\max_{l \in c_j} q_l \;\ge\; 0.7 \cdot \max_i \max_{l \in c_i} q_l. \tag{14}$$

If a cluster was dropped, take the clusters of $\{l : p_l \ge 0.12\}$ that overlap a kept cluster (the kept clusters themselves if there are none). Widen each by one link on both sides, decode p restricted to these regions with Rule 1, and add the result to the kept clusters.

Rule 3: site-block snap. The block catalogue of a panel counts how often each cluster b of a true onset set occurs in its train onset windows, $n(b)$. For every cluster c of the decoded set, let $t = \arg\max_{l \in c} p_l$ and let B_t be the catalogue blocks that contain t and whose links are all scored at T+30. With $b^* = \arg\max_{b \in B_t} n(b)$, replace c by b^* if

$$n(b^*) \ge 3 \qquad \text{and} \qquad n(b^*) \ge 0.75 \sum_{b \in B_t} n(b). \tag{15}$$

Rule 4: threshold of an ongoing window. A cell is queued when $\bar{p}_{h,l} \ge \theta_w$ with θ_w from Equation (10). The ratio r_w is the panel's queued share between 07:00 and 19:00 on the month's earlier days and on the window's own day up to T, divided by the same share in the training months. $\kappa_w = 1.5$ if the window starts exactly 360 minutes after the previous released window of its panel (spacing). $\kappa_w = 1$ if no link shows at least two queued observations between T−60 and T−10 minutes (a young queue). Otherwise $\kappa_w = 1.1$ (change point).

Final B: rolling recurrence. For an April window on day d, the in-month recurrence features of the eight members that use them are computed over the 16 days before d. Days before April 1 are taken from the end of March.

B. Implementation details

LightGBM members. Binary objective, learning rate 0.03, feature fraction 0.7, bagging fraction 0.8 (every round), L_2 penalty 5. Onset: 63 leaves, at least 100 rows per leaf, 600 rounds, seeds 42 and 43. Ongoing: 511 leaves, at least 40 rows per leaf, 800 rounds, seeds 42 and 43 (42–45 for the queue-focused family C, CP), no climatology columns. Per-panel members fit one model per panel with 255 leaves. The eleven members hold 144 boosters, 132 for ongoing and 12 for onset cells. The live-only onset model drops climatology, link position, panel identity, and all static link attributes.



Table 11 | **The U-Nets.** All share widths 32–256, crops of 64 links, batch 8, 250 steps per epoch, AdamW with weight decay 0.01 and an exponential moving average of the weights (0.999). "h31": the 31 held-out train days; fold k: the train days with index mod4 = k. Schedules are in epochs. Shipped models are part of the archive; the others are needed only to retrain.

model	seeds	held out	epochs	shipped	role
long runs 46, 48–52	46, 48–52	h31	382–1 000	✓	Task-1 ensemble, Task-2 fills
fold bases 0–3	60–63	fold k	148–191	–	bases of the forecasters
forecaster	62–65	fold k	144–153	✓	30-minute speed forecast
forecaster, L_1 loss	80–83	fold k	105–112	✓	30-minute speed forecast
forecaster, queue head	72–75	fold k	144–153	–	base of the queue-focused ones
queue-focused forecaster	90–93	fold k	110–113	✓	30-minute speed forecast
gap U-Nets 46, 48	146, 148	h31	43, 50	✓	long gaps after the queue windows
label denoisers	42–44	h31, Feb.	46–52	–	queue label field

Features. Per link, over the history $T-60, \ldots, T-5$ and the row at T: speed relative to v_{cut}, flow relative to capacity, occupancy and density (last value, means over 3, 6 and 12 steps, minima, maxima, slopes); queue flags (count, last value, time since queued); the same for up to three upstream and downstream neighbours; on- and off-ramp counts and their attachment; the geometry of the visible queue (component length, distances to head and tail, their slopes, numbers of queued links and steps); time of day, weekday and weekend; static link attributes (lanes, capacity per lane, free speed, length, jam density, capacity ratios to the neighbours); climatology of queueing at $T+5, \ldots, T+45$ by weekday type and of onsets by hour (onset model only); in-month recurrence (share of the month's earlier days queued within ±30 and ±10 minutes, any-day share, number of days); the horizon step h (ongoing); U-Net forecast speeds at the six steps (forecast families).

C. Compute

We trained all networks on a single Apple M2 Pro (32 GB, PyTorch MPS backend) with wall-clock budgets, and fitted the gradient-boosted models on its CPU (Table 12). The budgets add up to about 113 GPU hours for the 31 networks the final pipeline needs; the six state U-Nets take almost two thirds of it. Rebuilding both final submissions from the model archive takes 76 minutes on 8 CPU threads without a GPU (Figure 17).

Table 12 | **Compute behind the final submissions.** GPU times are the training budgets (wall-clock limits); the two 900-minute state runs stopped earlier at a cap of 1 000 epochs, so the total is an upper bound.

stage	hardware	time
state U-Nets (6; Task 1 and the history fills)	M2 Pro GPU	72 h
forecasters (16) and their fold bases (4)	M2 Pro GPU	38 h
label denoisers (3) and gap U-Nets (2)	M2 Pro GPU	3 h
LightGBM queue members (11, with 144 boosters)	10 CPU threads	2–30 min each
rebuild from the archive (`run_all.sh`)	8 CPU threads, no GPU	76 min

D. Notation

Table 13 | **Symbols and terms used in this report.**

symbol or term	meaning
T	origin of a Task-2 window: the last time stamp its forecast may use
$v_{\text{cut}} = 0.6\, v_f$	queue cut-off; a link is queued when its noise-free speed is at most v_{cut}
onset, ongoing	a window without, or with, a queue in its 60-minute history
spacing, change point, young	why an ongoing window was chosen (Section 2.3)
window unit	the weight of one window in S, $0.30/80 = 0.00375$
selector replica	our re-implementation of the published window-selection rule
forward test	fit on June–November, test on December–February
March replica	windows placed by the selector replica on the released March data
block ablation	a submission that replaces all onset or all ongoing windows of one panel, or of all panels, by an empty forecast
paired difference	the score difference between two submissions that differ in one change only
r_w, κ_w	congestion ratio and selection-kind factor of the ongoing threshold (Equation (10))

E. Use of AI tools

The author used AI tools to improve wording and to translate German drafts throughout the report.